DOSSIER DE RENDU FINAL · ITÉRATION 1 ET DÉBUT DE L'ITÉRATION 2

# Nancy Rock Vibration

Billetterie du festival : conception, première version de l'API et du site, puis comptes, commande, paiement, billets et backoffice

ÉQUIPE

Kamdine HASSANZADEH
Ilyass DGHISSI
Saad MOUBINE
Luca VIDIRI

MODULE

SAE « Atelier-projet de développement web 1 »
Université de Lorraine

VERSION

Rendu final
23 octobre 2026

# Sommaire

Ce dossier présente l'ensemble du travail : analyse du sujet, conception, maquettes, environnement technique et déploiement, API et site réalisés, sécurité, puis le **début de l'itération 2** (partie 9) et la conclusion. Chaque titre est cliquable.


1 **Contexte et objectifs** 3
- Le festival 3
- Fonctionnalités attendues 3
- Contraintes techniques 4

2 **Organisation du projet** 5
- Trello 5
- Discord 5
- Git 6

3 **Conception** 8
- 3.1 Scénarios d'usage 8
- 3.2 Règles métier 9
- 3.3 Diagramme de classes 10
- 3.4 Base de données 11
- 3.5 Contrat d'API 12

4 **Maquettes des écrans** 13
- 4.1 Programme du festival 13
- 4.2 Détail d'une soirée 14
- 4.3 Panier et Pass 3 jours 15
- 4.4 Connexion et paiement 16
- 4.5 Billets imprimables 17
- 4.6 Backoffice : suivi des jauges 18

5 **Environnement technique** 19
- Docker Compose 19
- Installation 19
- Déploiement sur docketu 19
- Jeu de données de test 21

6 **Backend réalisé** 22
- 6.1 Architecture hexagonale 22
- 6.2 Parcours d'une requête 24
- 6.3 Endpoints disponibles 24
- 6.4 Commande et paiement : une seule transaction 28
- 6.5 Tests unitaires 28

7 **Frontend réalisé** 30
- 7.1 Pages et parcours 30
- 7.2 Organisation du code JavaScript 33
- 7.3 Sass : architecture et bibliothèque d'outils 34
- 7.4 Grid et Flexbox : quand et pourquoi 35
- 7.5 Adaptation au terminal 36
- 7.6 Qualité HTML et vérifications 37

8 **Sécurité** 38
- 8.1 Injections SQL 38
- 8.2 Validation des entrées 39
- 8.3 Côté site : XSS et en-têtes HTTP 39
- 8.4 Comptes et jetons 41
- 8.5 Commande et paiement 41
- 8.6 Tests d'attaque 42

9 **Début de l'itération 2** 43
- 9.1 Étape 5 : comptes et Pass 3 jours 44
- 9.2 Étape 6 : commande et paiement 45
- 9.3 Étape 7 : billets et backoffice 46
- 9.4 Répartition du travail 47
- 9.5 Suite de l'itération 2 47

10 **Conclusion** 48

# 1 Contexte et objectifs

**NRV.net** est l'application de billetterie du festival **Nancy Rock Vibration**. Elle permet de consulter le programme, d'acheter des billets en ligne et, pour l'organisation, de suivre le remplissage des soirées.

## Le festival

- Le festival dure **2 semaines**. Chaque **soirée** a lieu à une date unique, dans un lieu donné, avec une jauge (nombre maximal de places).
- Une soirée regroupe **2 à 3 spectacles**. Chaque spectacle a **un seul style** de musique (classic rock, blues rock, métal, chanson…), une soirée peut mélanger les styles.
- Un **billet** donne accès à tous les spectacles de sa soirée. Deux tarifs : **normal** et **réduit** (étudiants, demandeurs d'emploi, personnes en situation de handicap).
- Le **Pass 3 jours** est un billet spécifique qui donne accès à **3 soirées distinctes**, avec une remise de 3 %. Il existe aussi en normal et en réduit.
- L'achat se fait uniquement en ligne, après inscription, et seulement s'il reste des places. Le billet n'est pas nominatif : il est accessible par une URL contenant un jeton aléatoire.

## Fonctionnalités attendues

| Réf. | Fonctionnalité du sujet | État au rendu |
|---|---|---|
| F1 | Liste des spectacles : titre, date, horaire, image | **Réalisé (API + site)** |
| F2 | Détail d'une soirée : nom, thématique, date, lieu, tarifs, spectacles (artistes, description, style, vidéo) | **Réalisé (API + site)** |
| F3 | Clic sur un spectacle → détail de sa soirée | **Réalisé (lien HATEOAS)** |
| F4 · F5 | Filtrage de la liste par date et par style | **Réalisé (API + site)** |
| F6 · F7 | Ajout au panier, état du panier et total | **Réalisé (site)** |
| F8 · F9 | Validation en commande (compte obligatoire), paiement simulé avec contrôle et mise à jour des places | **Réalisé (itération 2)** |
| F10 | Billets imprimables (nom de l'acheteur, référence, date, catégorie) | **Réalisé (itération 2)** |
| F11 | Backoffice : places vendues et recette par soirée | **Réalisé (itération 2)** |
| E1 | Pagination de la liste des spectacles | **Réalisé (API + site)** |
| E2 · E4 | Modification du panier, vérification des places à l'ajout | **Réalisé (site)** |
| Pass | Pass 3 jours : 3 soirées distinctes, tarif normal ou réduit | **Réalisé (itération 2)** |
| E3 | Backoffice : ajout de spectacles et de soirées | **Réalisé (itération 2)** |

## Contraintes techniques

### Backend

- PHP 8 / Slim 4 dans des conteneurs Docker
- API RESTful en JSON, liens HATEOAS
- Architecture hexagonale stricte, injection de dépendances (PHP-DI)
- PDO avec requêtes préparées, validation des données
- PSR-1 et PSR-4, tests unitaires (PHPUnit)

### Frontend

- HTML5 sémantique, valide W3C
- Sass/SCSS, responsive (mobile, tablette, ordinateur)
- CSS Grid et Flexbox, feuille `@media print` pour le billet
- Données chargées par `fetch` et affichées avec Handlebars

L'application doit être déployée dans des conteneurs Docker sur **docketu.iutnc.univ-lorraine.fr**.

# 2 Organisation du projet

## Trello

L'équipe a organisé le travail sur un tableau **Trello** partagé : une colonne par étape (conception, environnement, production, déploiement), une carte par tâche, avec les membres assignés (K Kamdine, ID Ilyass, L Luca, SM Saad). Une carte est cochée quand elle est terminée ; chaque carte correspond à un ou plusieurs commits.

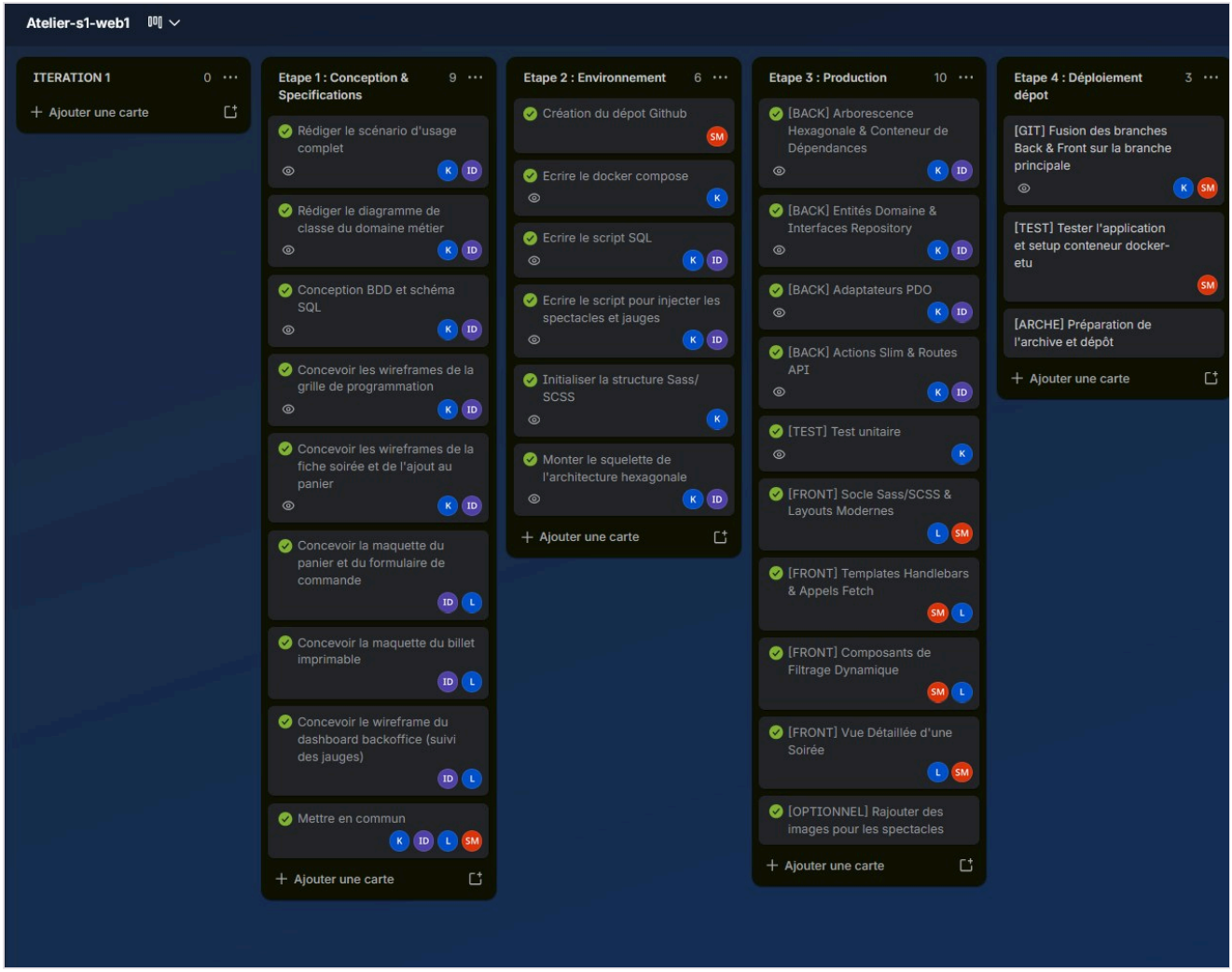



*Figure 1 · Tableau Trello de l'itération 1 (état au 21 octobre 2026)*

Les cartes [BACK] et [FRONT] sont préfixées comme les messages de commit. L'étape 4 (fusion des branches sur `main`, tests sur docketu, archive de rendu) clôt l'itération.

## Discord

Au quotidien, l'équipe a communiqué sur un **serveur Discord** : questions et entraide pendant le développement, points d'avancement en fin de séance, répartition des cartes, partage de captures d'écran et de liens, et annonce de chaque fusion sur `main` pour que chacun récupère la dernière version. Le Trello garde la trace de ce qui est décidé ; Discord sert à en discuter.

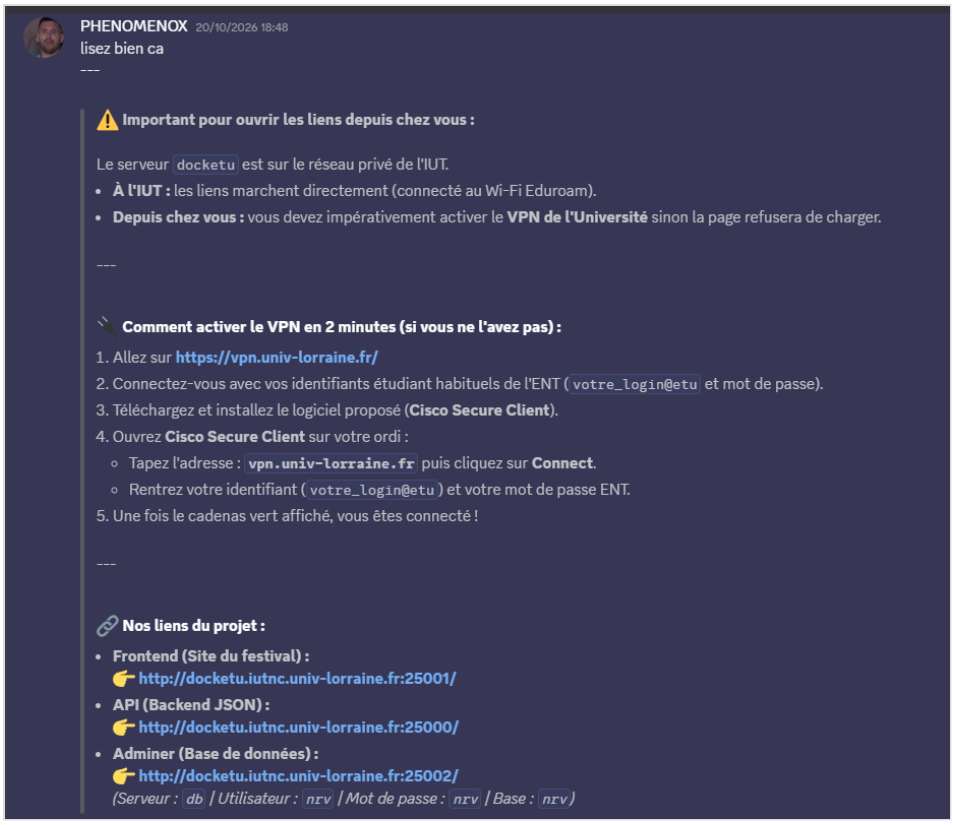



*Figure 2 · Message sur le Discord de l'équipe après le premier déploiement : accès par le VPN et liens de la version en ligne*

## Git

- Dépôt du groupe : **Atelier-Projet-de-developpement-web-1**. Chacun travaille sur sa branche (`production-kam`, `ilyass`, `etp7-luca`, `production-saad`) ; les branches sont fusionnées sur `main`, qui est la version rendue et déployée.
- Un commit par carte, dont le message reprend le nom de la carte : `2.1 docker-compose et config`, `[back] adaptateurs pdo`…
- Les commentaires du code sont courts et rédigés en français.

### Historique de l'itération 1

```
2.1 docker-compose et config
2.2 seed fixtures
2.3 arborescence hexagonale
1.3 schema bdd pass 3 jours en billet
2.2 seed fixtures avec pass 3 jours
initialiser la structure sass/scss
fix volumes docker pour selinux (fedora)
fix composer : php-jwt 7 (faille de securite)
[back] entites domaine et interfaces repository
[back] adaptateurs pdo
2.2 seed : groupes en plus pour eviter les noms
de secours
[back] actions slim et routes api
[back] arborescence hexagonale et conteneur de
dependances
[back] ports de sortie dans application/port/out
tests unitaires phpunit du domaine et des cas
d'usage
preparer le deploiement docketu (ports, cors,
url de l'api)
readme : installation, donnees de test, api,
tests, deploiement
1.5 maquettes visuelles des ecrans
document de rendu iteration 1 et diagrammes uml
readme simplifie
[front] socle sass/scss et layouts
[front] templates handlebars et appels fetch
[front] composants de filtrage dynamique
[front] vue detaillee d'une soiree
[front] images locales et videos reelles dans le
seed
[front] page panier, navigation et bouton video
securite : validation des parametres et en-tetes
http
```

Le Trello et la répartition du travail de l'itération 2 sont en partie 9.

La documentation accompagne le code dans le dépôt : `README.md` (installation, données de test, API, tests, déploiement), `docs/arborescence.md`, `docs/maquettes/`, `docs/uml/` (sources des diagrammes) et `sql/01_schema.sql`.

# 3 Conception

## 3.1 Scénarios d'usage

Trois acteurs utilisent l'application : le **visiteur** (anonyme, il consulte), l'**acheteur** (inscrit, il commande et paie) et l'**administrateur** (il suit les jauges).

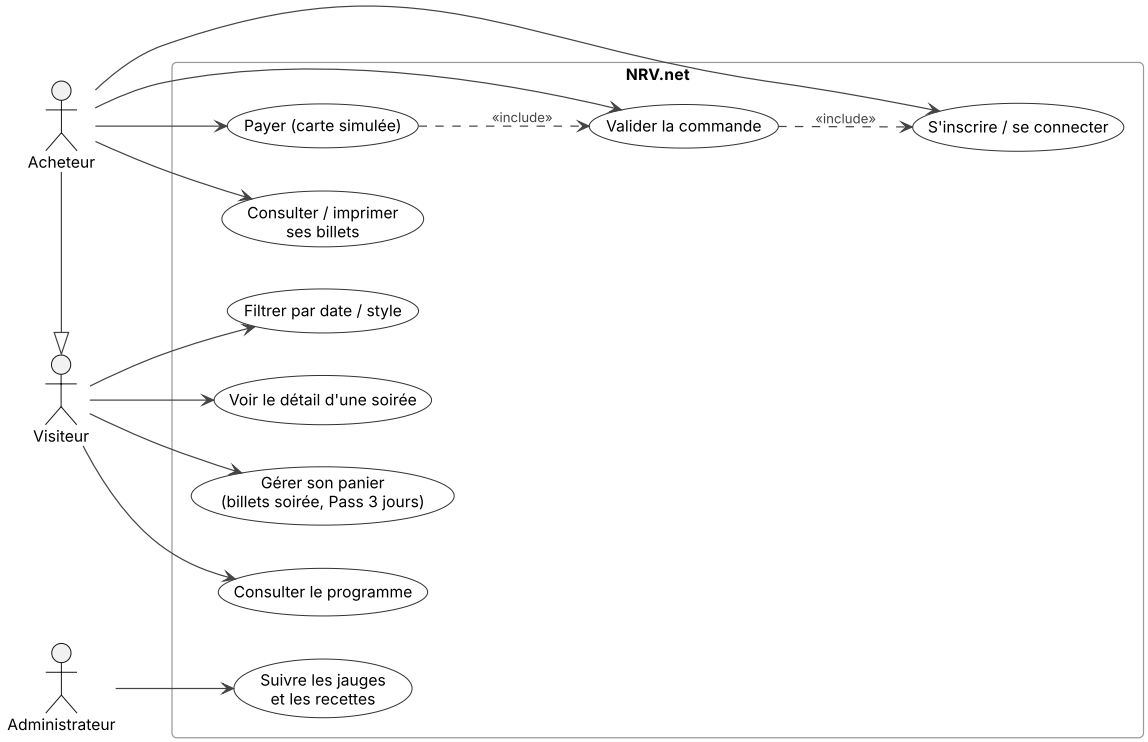



*Figure 3 · Cas d'utilisation de NRV.net*

**Parcours A · Léa découvre le programme (visiteur, sur téléphone)**

1. Elle ouvre l'accueil : la grille des spectacles s'affiche (image, titre, date, horaire), 12 par page.
2. Elle choisit le style « Métal » puis la date du 11 juin : la grille se met à jour sans recharger la page.
3. Elle touche une carte : le détail de la soirée s'ouvre (lieu, tarifs, places restantes, les 3 spectacles avec leur vidéo).

**Parcours B · Marc achète des billets (acheteur, sur ordinateur)**

1. Sur « Nuit métal », il ajoute 2 billets au tarif normal. Le site vérifie qu'il reste assez de places.
2. Dans le panier, il compose un **Pass 3 jours** réduit pour sa fille (11, 12 et 15 juin) : 58,20 € au lieu de 60 €.
3. Il valide le panier, se connecte (ou s'inscrit) : la commande est créée au statut « en attente ».
4. Il paie avec une carte (simulée). Le serveur vérifie les places **au moment du paiement**, les réserve et crée les billets, chacun avec un jeton unique.
5. Il imprime ses billets : une page A4 sobre, sans menu, grâce à la feuille `@media print`.

Cas d'erreur prévus : carte invalide (422), soirée devenue complète entre-temps (409, rien n'est réservé), double clic sur « Payer » (409, pas de billet en double), session expirée (401).

### Parcours C · Sophie suit les ventes (administratrice)

Elle ouvre le backoffice : pour chaque soirée, la jauge, les places vendues (billets et pass), les places restantes, le taux de remplissage et la recette. Les soirées complètes ou presque complètes sont mises en évidence.

## 3.2 Règles métier

| Règle | Comment elle est appliquée |
|---|---|
| Une soirée a 2 à 3 spectacles | l'entité `Soiree` refuse les autres cas (`NombreSpectaclesInvalideException`) |
| Un spectacle a un seul style | clé étrangère `spectacles.style_id` ; l'entité `Spectacle` reçoit un seul `Style` |
| Tarifs et jauge portés par la soirée | colonnes `tarif_normal`, `tarif_reduit`, `jauge`, `places_vendues` ; tarif réduit ≤ normal ; jauge ≤ capacité du lieu |
| On ne vend pas au-delà de la jauge | `Soiree::reserver()` lève `SoireeCompleteException` ou `PlacesInsuffisantesException` ; contrainte `CHECK` en base en plus |
| Pass 3 jours = un billet spécifique | formule `pass` en base (87,30 € / 58,20 €) ; billet de type `PASS` relié à 3 soirées distinctes ; il prend une place dans chacune |
| Places contrôlées au paiement | transaction : verrouillage des soirées, contrôle, réservation, création des billets ; tout est annulé au premier refus |
| Billet non nominatif, accessible par URL | jeton UUID v4 aléatoire unique ; le nom de l'acheteur est seulement imprimé |
| Montants exacts | tous les prix sont stockés en centimes (entiers), pas de flottants |

## 3.3 Diagramme de classes

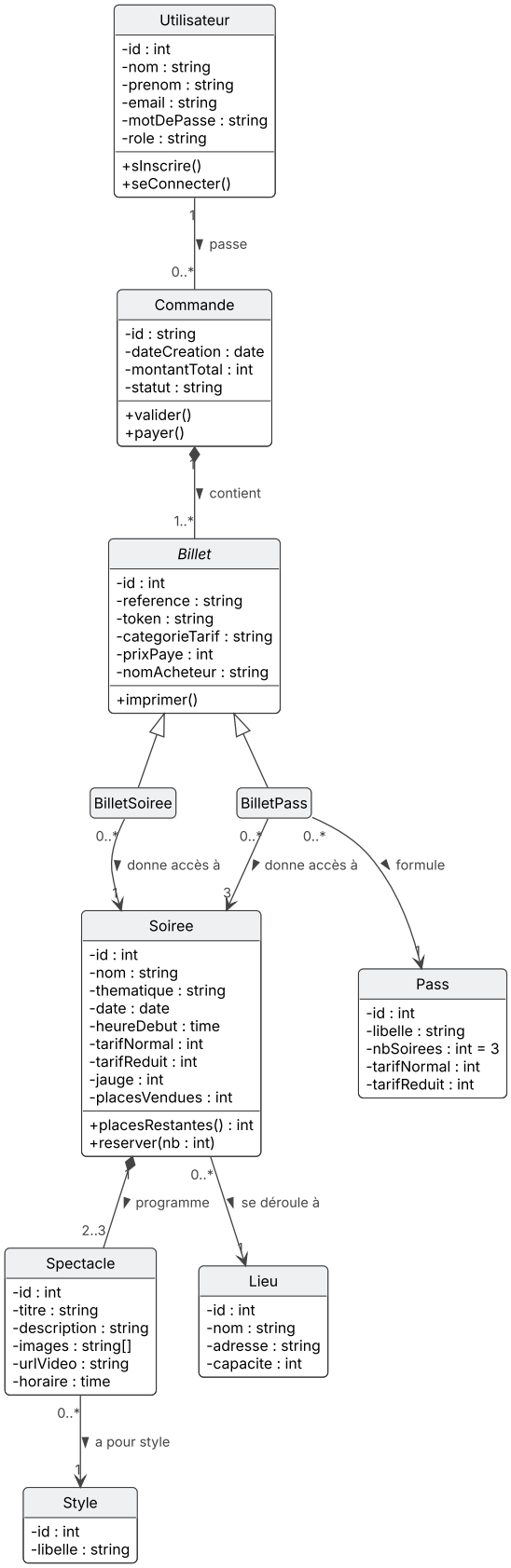



*Figure 4 · Modèle du domaine*

### Comment le lire

- **Utilisateur → Commande → Billet** : un utilisateur passe des commandes ; une commande contient ses billets (composition, losange plein).
- **Billet** est abstrait, avec deux sortes :
  - **BilletSoiree** donne accès à **1** soirée ;
  - **BilletPass** donne accès à **3** soirées et suit une formule **Pass** (tarif normal ou réduit).
- **Soiree** porte les tarifs, la jauge et les places vendues ; elle *programme* 2 à 3 **Spectacle** (composition) et *se déroule* dans un **Lieu**.
- Chaque **Spectacle** a exactement **un Style**.

> **Correspondance en base**
>
> En base, l'héritage `Billet` est représenté par une seule table `billets` avec une colonne `type` (SOIREE ou PASS). Les 3 soirées d'un pass sont dans la table `billet_pass_soirees`.

> **Choix de modélisation**
>
> - le Pass 3 jours est un billet à part entière, et non une remise calculée sur le panier ;
> - le style est une classe, reliée à chaque spectacle par une multiplicité 1 ;
> - le panier n'est pas une classe : il vit dans le navigateur et n'est pas enregistré ;
> - le paiement simulé est une méthode de `Commande`.

## 3.4 Base de données

MariaDB 10.11, 14 tables InnoDB en `utf8mb4`. Le script complet est `sql/01_schema.sql` ; il est joué automatiquement au premier démarrage du conteneur.

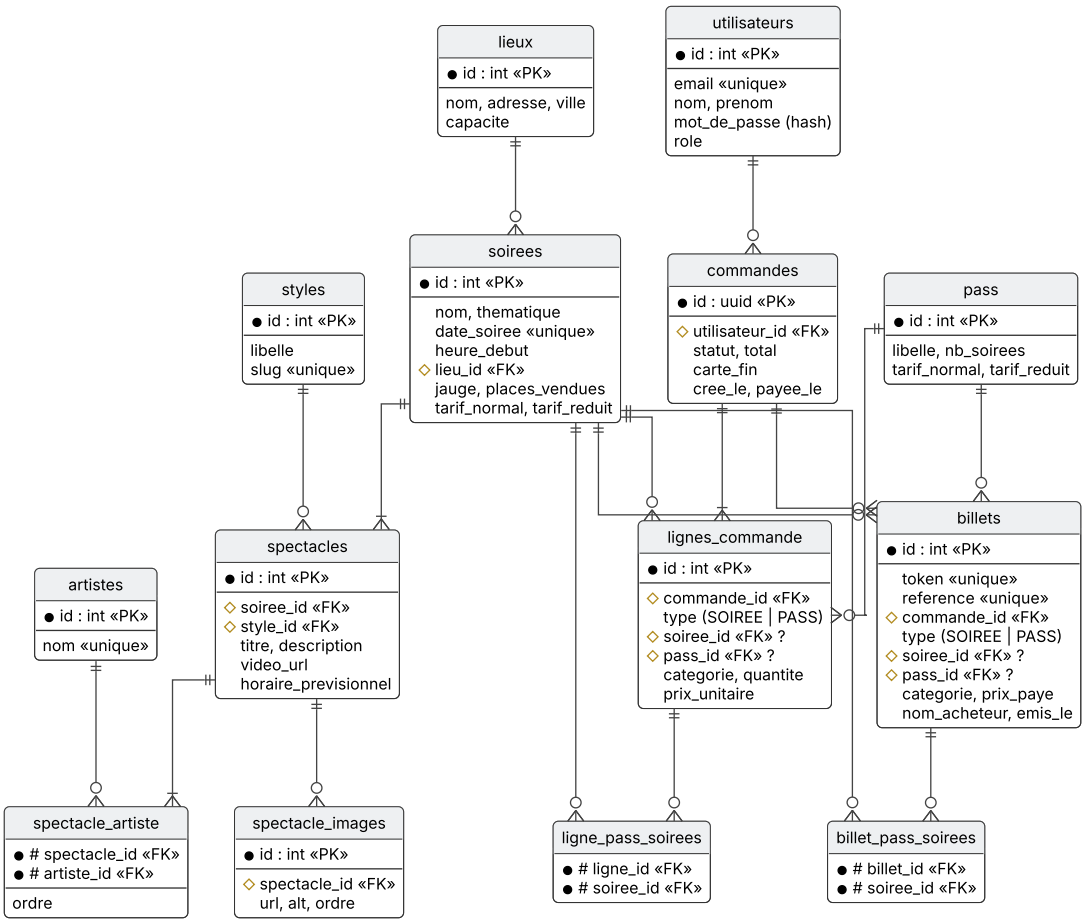



*Figure 5 · Schéma relationnel (PK clé primaire, FK clé étrangère, ? = facultatif)*

| Tables | Rôle |
|---|---|
| `lieux`, `styles`, `artistes` | données de référence |
| `soirees`, `spectacles` | le programme ; `soirees.places_vendues` est le compteur de la jauge |
| `spectacle_artiste`, `spectacle_images` | un spectacle a un ou plusieurs artistes et une ou plusieurs images |
| `pass` | la formule Pass 3 jours et ses deux tarifs |
| `utilisateurs`, `commandes`, `lignes_commande`, `ligne_pass_soirees` | les comptes et les commandes ; une ligne = n billets identiques (soirée ou pass) |
| `billets`, `billet_pass_soirees` | les billets émis au paiement, avec leur jeton unique |
| vue `v_jauges_soirees` | pour le backoffice : jauge, vendues, restantes, taux, recette (un pass compte pour un tiers dans chacune de ses soirées) |

Contraintes notables : une soirée par date ( `UNIQUE` ), `places_vendues ≤ jauge` , tarif réduit ≤ normal, cohérence type / soirée / pass sur les billets ( `CHECK` ), jeton et référence uniques.

## 3.5 Contrat d'API

Toutes les réponses sont en JSON. Une ressource : `{"type":"resource", "<nom>":{…}, "links":{…}}` ; une liste : `{"type":"collection", "count", "total", …, "links"}`. Les erreurs suivent le format RFC 7807 ( `application/problem+json` ).

| Verbe | Route | Rôle | État |
|---|---|---|---|
| GET | `/spectacles` | liste paginée, filtres `date` et `style` | Réalisé |
| GET | `/spectacles/{id}` | un spectacle | Réalisé |
| GET | `/soirees/{id}` | soirée complète avec ses spectacles | Réalisé |
| GET | `/styles` , `/dates` | valeurs des filtres | Réalisé |
| GET | `/soirees` | les 14 soirées et leurs places restantes | Réalisé |
| GET | `/pass` | formule Pass 3 jours et ses tarifs | Réalisé |
| POST | `/auth/register` , `/auth/signin` | inscription, connexion (jeton JWT) | Réalisé |
| GET | `/compte` | le compte connecté | Réalisé |
| POST | `/commandes` | panier → commande « en attente » | Réalisé |
| GET | `/commandes/{id}` | une de mes commandes (récapitulatif avant paiement) | Réalisé |
| POST | `/commandes/{id}/paiement` | paiement simulé, réservation des places, création des billets | Réalisé |
| GET | `/commandes` , `/commandes/{id}/billets` , `/billets` | mes commandes et mes billets | Réalisé |
| GET | `/billets/{token}` | un billet, par son jeton | Réalisé |
| GET | `/admin/jauges` | jauges et recettes (rôle admin) | Réalisé |
| GET · POST | `/admin/referentiels` , `/admin/soirees` , `/admin/soirees/{id}/spectacles` | ajout de soirées et de spectacles (rôle admin, E3) | Réalisé |

Formats : montants en centimes ( `3000` = 30,00 €), dates ISO ( `2027-06-11` ), heures `HH:MM` .

# 4 Maquettes des écrans

Les maquettes sont des pages HTML statiques (`docs/maquettes/`) réalisées avec la feuille de style Sass du projet et les données du jeu de test. Elles fixent la structure HTML, la mise en page responsive et les composants de chaque écran. Tous les écrans maquettés sont maintenant réalisés et branchés sur l'API : Programme, Soirée et Panier à l'itération 1 (partie 7), Connexion, Paiement, Billets et Backoffice au début de l'itération 2 (partie 9).

## 4.1 Programme du festival

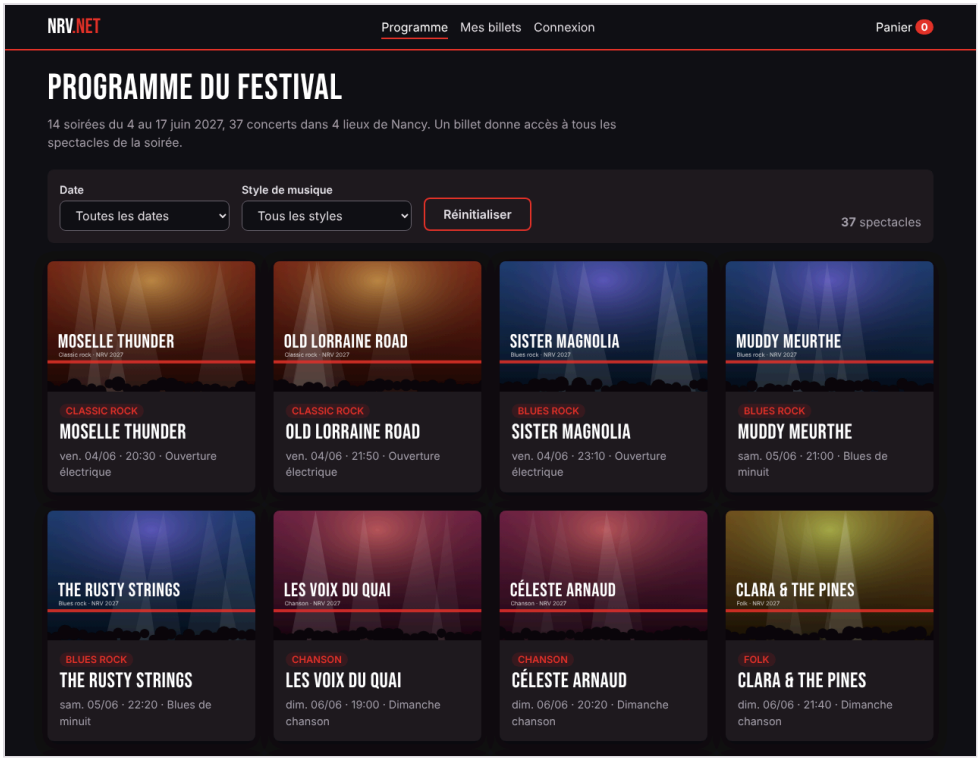



*Figure 6 · Programme sur ordinateur : filtres, grille, pagination*

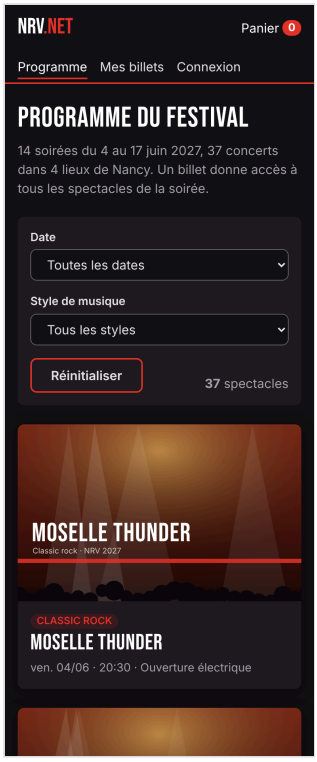



*Figure 7 · Sur téléphone*

Chaque carte est un lien vers la soirée (F3). Les filtres date et style se cumulent (F4, F5) et la grille se met à jour sans rechargement ; la pagination montre 12 spectacles par page (E1).

## 4.2 Détail d'une soirée

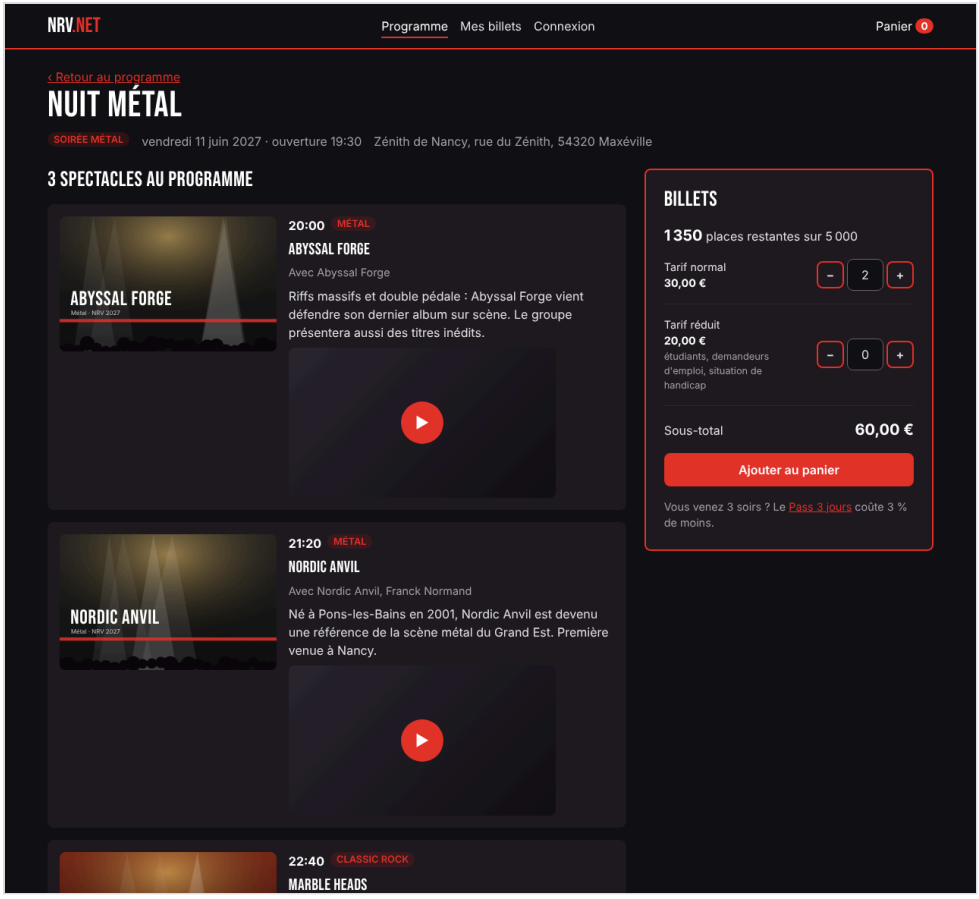



*Figure 8 · Soirée « Nuit métal » : 3 spectacles et encart d'achat*




*Figure 9 · Sur téléphone*

L'encart d'achat reste visible pendant le défilement sur ordinateur (`position: sticky`). Il affiche les places restantes, ce qui permet de bloquer un ajout impossible (E4).

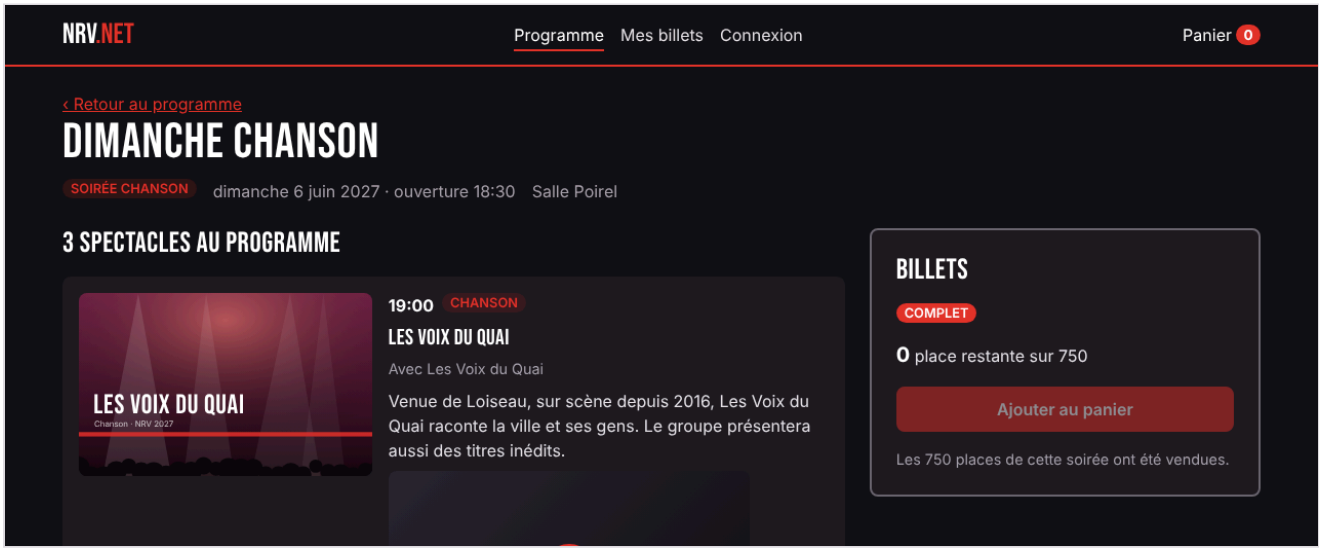



*Figure 10 · Soirée complète : le bouton d'achat est désactivé*

## 4.3 Panier et Pass 3 jours

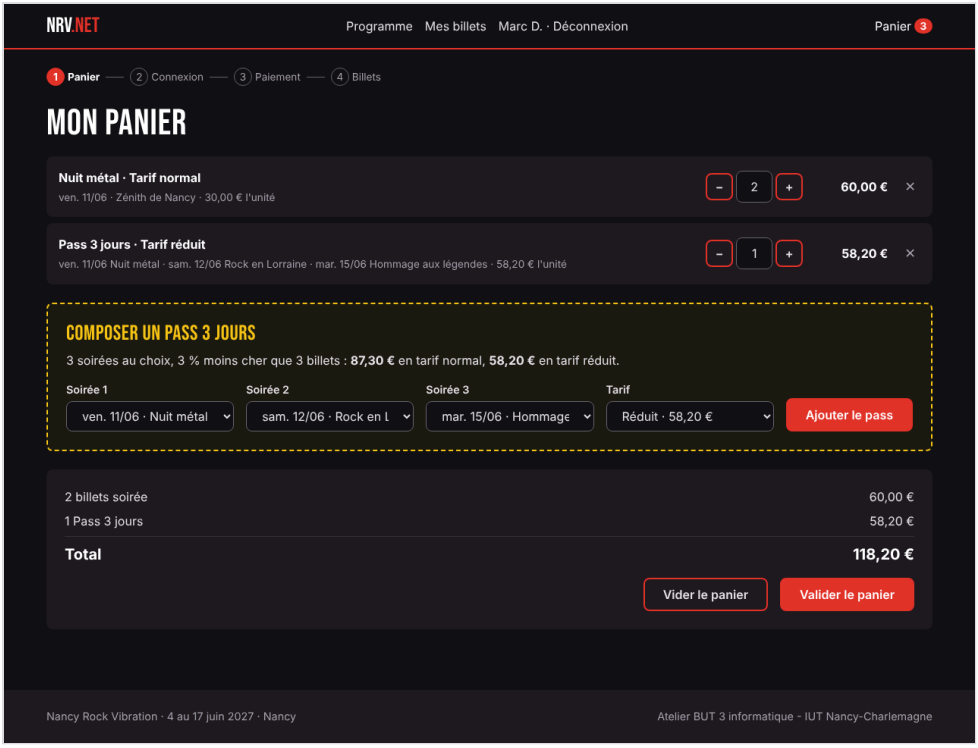



*Figure 11 · Panier : billets soirée, Pass 3 jours, total*




*Figure 12 · Sur téléphone*

Chaque ligne se modifie avec − et + (E2). L'encart « Composer un Pass 3 jours » propose 3 soirées et le tarif ; le pass devient une ligne du panier.

## 4.4 Connexion et paiement

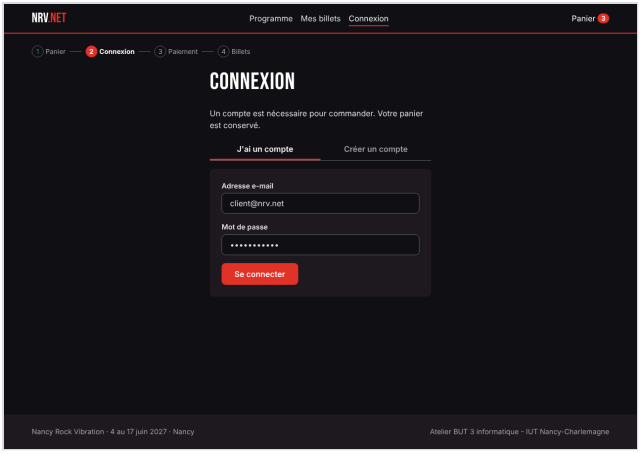



*Figure 13 · Connexion (compte obligatoire pour commander)*

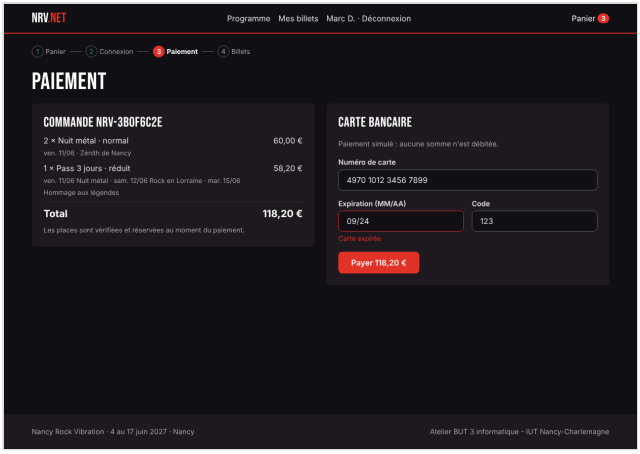



*Figure 14 · Paiement : récapitulatif et carte, avec un message d'erreur*

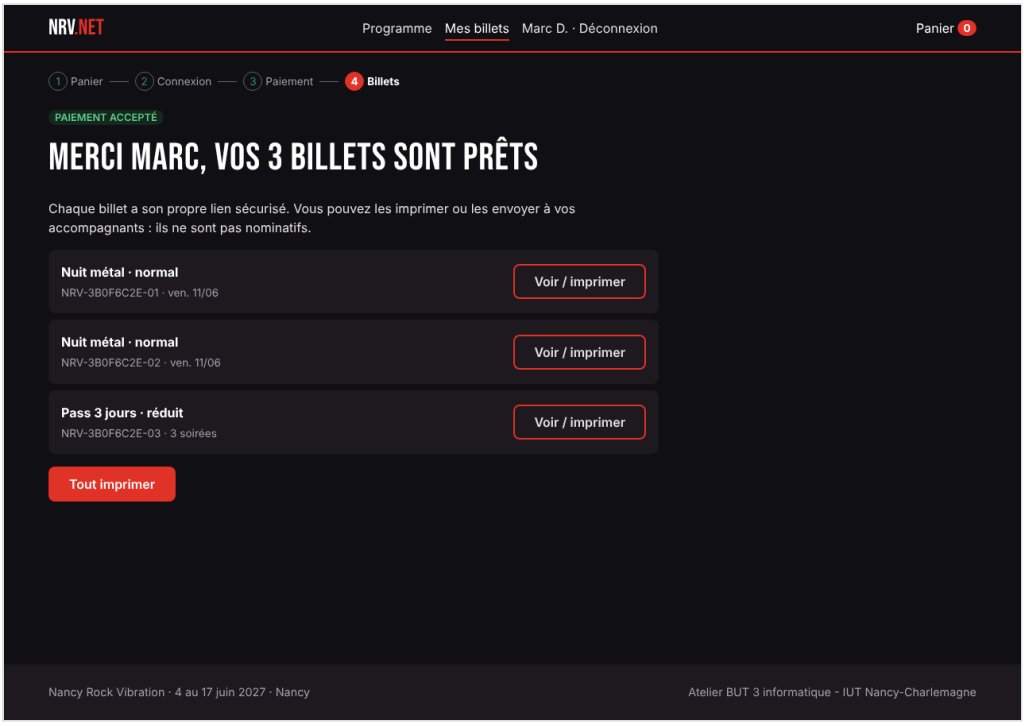



*Figure 15 · Confirmation : un lien par billet*

## 4.5 Billets imprimables

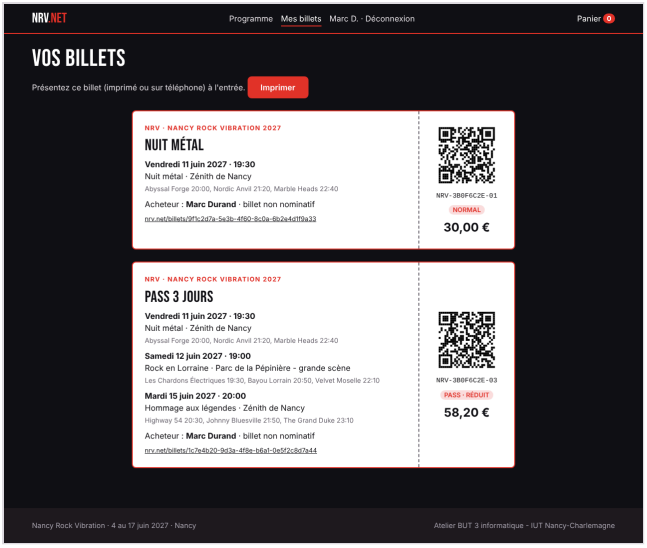



*Figure 16 · Billet soirée et Pass 3 jours à l'écran*

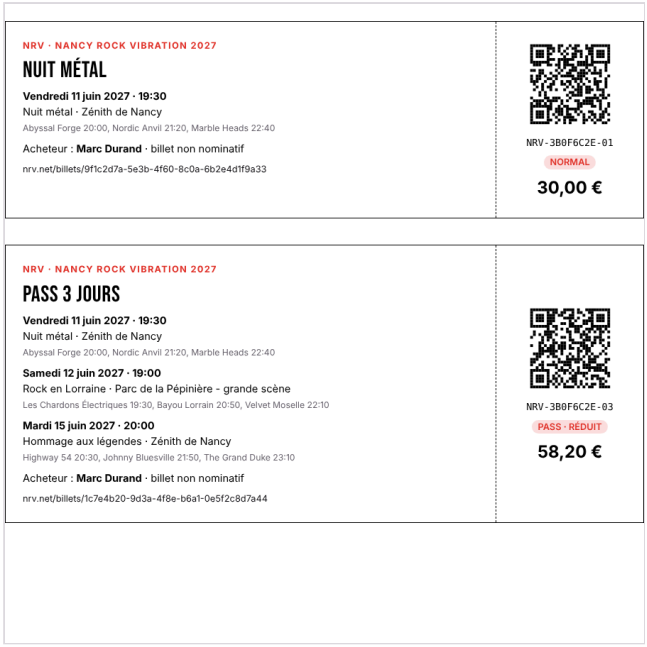



*Figure 17 · Rendu à l'impression (`@media print`)*

Chaque billet porte le nom de l'acheteur, la référence, la ou les soirées (date, horaire, lieu, spectacles), la catégorie et le prix (F10). À l'impression : en-tête, menu et boutons disparaissent, le billet est en noir et blanc, et il ne se coupe jamais entre deux pages.

## 4.6 Backoffice : suivi des jauges

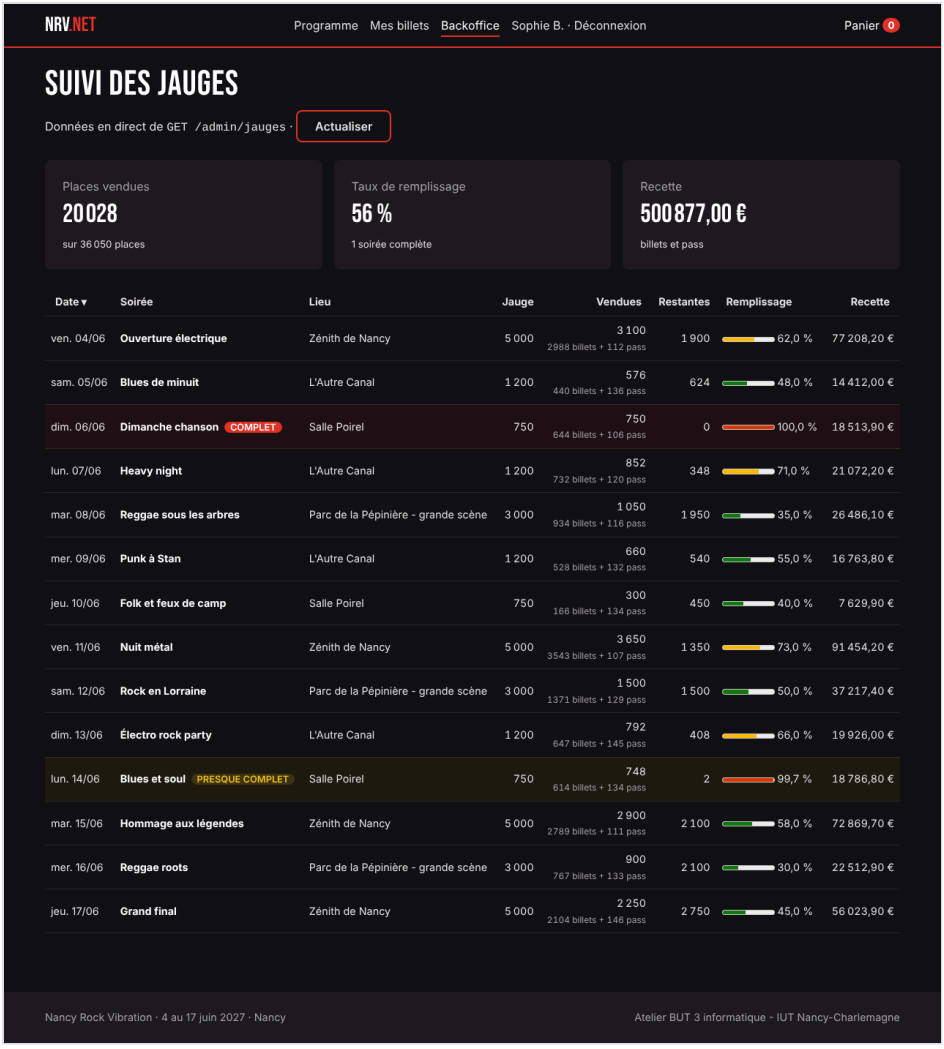



*Figure 18 · Jauge et recette de chaque soirée (F11)*

Trois indicateurs en haut (places vendues, taux de remplissage, recette), puis une ligne par soirée. Les soirées complètes sont en rouge, celles à plus de 90 % en jaune. Sur téléphone, le tableau défile horizontalement.

### Choix de mise en page

- **CSS Grid** pour les zones en deux dimensions : grille des spectacles ( `repeat(auto-fill, minmax(16rem, 1fr))` : 1 colonne sur téléphone, 4 sur ordinateur), page soirée (spectacles | achat), paiement (récapitulatif | carte), indicateurs du backoffice.
- **Flexbox** pour les alignements sur une ligne : en-tête et navigation, barre de filtres, lignes du panier, sélecteurs de quantité, étapes de la commande.
- Trois tailles d'écran (mixin Sass `mq()` ) : mobile par défaut, tablette dès 600 px, ordinateur dès 1024 px.

# 5 Environnement technique

## Docker Compose

| Service | Image | Adresse | Rôle |
|---|---|---|---|
| `db` | mariadb:10.11 | 127.0.0.1:3307 | base de données ; le schéma est joué au premier démarrage |
| `php` | php:8.3-fpm (Dockerfile) | interne | exécute l'API Slim ; contient Composer |
| `api` | nginx:1.27 | localhost:8080 | serveur web de l'API (tout passe par `public/index.php`) |
| `front` | nginx:1.27 | localhost:8081 | site statique (`front/public`) |
| `adminer` | adminer:4 | localhost:8082 | interface pour consulter la base |
| `sass` | node:20 (profil dev) | — | recompile le Sass à chaque sauvegarde |

Les ports se changent dans le fichier `.env` (utile sur un serveur partagé). Sur Fedora, les volumes ont l'option `:z` pour SELinux.

## Installation

```
git clone https://github.com/Kamdinee/Atelier-Projet-de-developpement-web-1.git
cd Atelier-Projet-de-developpement-web-1
git checkout main
cp .env.example .env

docker compose up -d --build
docker compose exec php composer install
docker compose exec php composer seed     # jeu de donnees de test
docker compose exec php composer test     # tests unitaires
```

## Déploiement sur docketu

L'application est déployée sur le serveur Docker de l'IUT. Il est sur le réseau privé de l'université : depuis l'IUT (Wi-Fi Eduroam) les liens marchent directement ; depuis l'extérieur, il faut d'abord activer le **VPN de l'Université de Lorraine** (Cisco Secure Client, adresse `vpn.univ-lorraine.fr`, identifiants ENT `login@etu`).

Le déploiement reprend exactement l'installation locale : le même dépôt et le même `docker-compose.yml`, seul le fichier `.env` change. Les étapes suivies :

1. Connexion au serveur docketu en SSH avec les identifiants de l'IUT (depuis l'IUT ou par le VPN), puis `git clone` du dépôt et passage sur `main`.
2. Création du `.env` à partir de `.env.example` : ports libres sur le serveur partagé (`API_PORT=25000`, `FRONT_PORT=25001`, `ADMINER_PORT=25002`, `DB_PORT_HOTE=25003`), `APP_ENV=prod` pour masquer le détail des erreurs, un `JWT_SECRET` aléatoire de plus de 32 caractères, et

`CORS_ORIGINS=http://docketu.iutnc.univ-lorraine.fr:25001` pour que seul notre site puisse appeler l'API.

3. Port de l'API reporté dans `front/public/js/config.js` (`API_PORT = 25000`) : le site appelle l'API sur la même machine, à ce port.
4. `docker compose up -d --build`, puis `composer install` et `composer seed` dans le conteneur `php` : la base est créée par le script SQL au premier démarrage, puis remplie avec le jeu de test.
5. Vérification en ligne : pages du site, réponses de l'API, connexion avec les comptes de test.

Pour une nouvelle version : `git pull` puis `docker compose up -d --build`. Deux problèmes propres au serveur ont été corrigés au premier déploiement : la politique de sécurité (CSP) du site bloquait les appels vers l'API, qui n'est pas sur le même port (elle autorise maintenant l'hôte du site sur tous ses ports) ; Adminer redémarrait en boucle car docketu n'a pas d'IPv6 (il écoute maintenant en IPv4).

| Service | Adresse | Accès |
|---|---|---|
| Site du festival | `http://docketu.iutnc.univ-lorraine.fr:25001/` | — |
| API (JSON) | `http://docketu.iutnc.univ-lorraine.fr:25000/` | — |
| Adminer (base) | `http://docketu.iutnc.univ-lorraine.fr:25002/` | serveur `db`, utilisateur et mot de passe `nrv`, base `nrv` |

## Jeu de données de test

Le script `api/bin/seed.php` (FakerPHP, graine fixe : les données sont toujours les mêmes) remplit la base avec :

- 4 lieux à Nancy, 8 styles, 14 soirées du 4 au 17 juin 2027, 37 spectacles, 43 artistes ;
- une photo de concert par spectacle (Unsplash, Pexels, Pixabay), servie par le site ( `img/spectacles/` ) et préparée par `front/tools/photos.py` , et une vraie vidéo YouTube par style ;
- la formule Pass 3 jours (87,30 € / 58,20 €) ;
- 42 utilisateurs, environ 5 600 commandes payées, 18 200 billets soirée et 600 Pass 3 jours, soit environ 503 000 € de recette.

| Cas de test remarquables | Id | Situation |
|---|---|---|
| Dimanche chanson (6 juin, Salle Poirel) | 3 | **complète** : tout achat doit être refusé |
| Blues et soul (14 juin, Salle Poirel) | 11 | **2 places restantes** : 2 billets passent, 3 sont refusés |
| Nuit métal (11 juin, Zénith) | 8 | 3 spectacles, grande jauge |

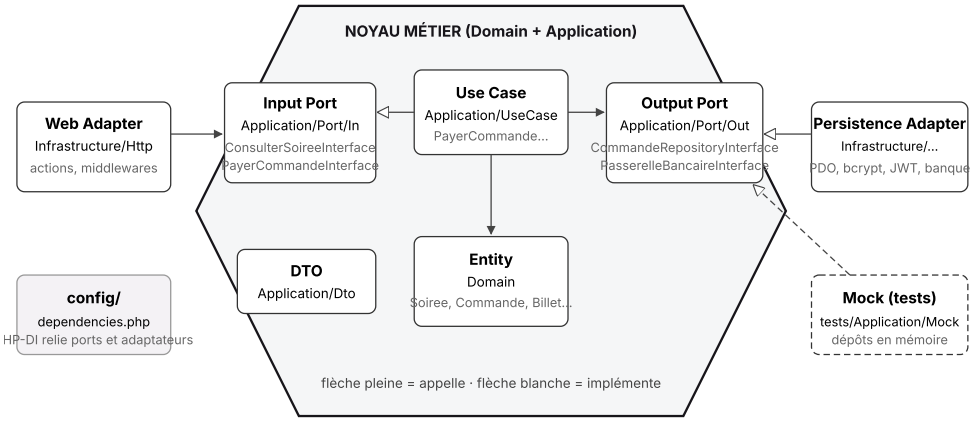
| Compte | E-mail | Mot de passe |
|---|---|---|
| Administratrice | admin@nrv.net | admin12345 |
| Client | client@nrv.net | client12345 |
| 40 clients fictifs | voir la table `utilisateurs` | motdepasse1 |

# 6 Backend réalisé

L'itération 1 a livré la partie « consultation » de l'API (programme, filtres, pagination, détail des soirées). Le début de l'itération 2 y ajoute les comptes, la commande, le paiement simulé, les billets et le backoffice. L'API compte aujourd'hui **22 routes**, 16 cas d'usage, et le noyau métier est couvert par **159 tests unitaires**.

## 6.1 Architecture hexagonale

Le code suit le schéma vu en cours : un **noyau métier** (entités, cas d'usage, ports) qui ne dépend de rien, entouré d'**adaptateurs**. Les ports sont des **interfaces** ; les cas d'usage sont des **services**.




> **Inversion de dépendances**
>
> Le cas d'usage `ConsulterSoiree` demande une soirée à une **interface** ( `SoireeRepositoryInterface` ) sans savoir qu'il y a PDO derrière. C'est `config/dependencies.php` qui branche l'implémentation PDO (inversion de dépendances). Dans les tests, on branche un faux dépôt en mémoire à la place, sans rien changer au cas d'usage. Le même principe sert pour tout ce qui sort du noyau à l'itération 2 : la banque ( `PasserelleBancaireInterface` , simulée par `PaiementSimule` ), la transaction SQL, le hachage des mots de passe, le jeton JWT, l'heure et la génération des identifiants sont des ports de sortie, remplacés par des faux dans les tests.

| Couche | Dossier | Contient · dépend de |
|---|---|---|
| Domaine | `src/Domain` | entités ( `Soiree` , `Spectacle` , `Lieu` , `Style` , `Pass` , `Utilisateur` , `Commande` , `Billet` ), value objects ( `Tarifs` , `LigneCommande` , `CarteBancaire` , `JetonBillet` , énumérations `CategorieTarif` , `StatutCommande` , `Role` ...), exceptions métier · **ne dépend de rien** (PHP natif seulement) |
| Application | `src/Application` | 16 ports d'entrée et 15 ports de sortie (interfaces), 16 cas d'usage (services), DTO · dépend du domaine seulement |

| Couche | Dossier | Contient · dépend de |
| --- | --- | --- |
| Adaptateur d'entrée | `src/Infrastructure/Http` | actions Slim, validation des données reçues, middlewares (CORS, connexion, administrateur), erreurs JSON, liens HATEOAS · appelle les ports d'entrée avec des DTO |
| Adaptateurs de sortie | `Persistence/Pdo`, `Security`, `Token`, `Payment`, `Clock` | dépôts et transaction PDO, hachage bcrypt, jetons JWT et UUID, banque simulée, horloge · implémentent les ports de sortie |

## 6.2 Parcours d'une requête

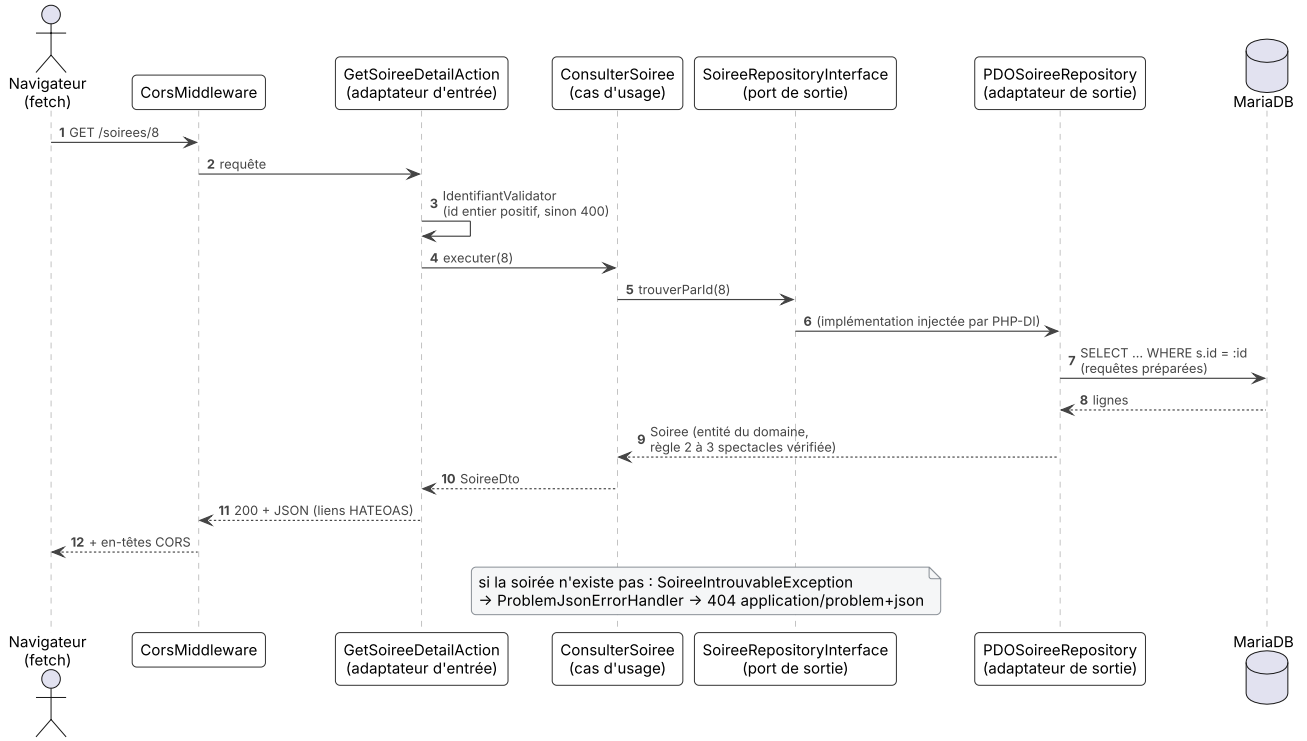



*Figure 19 · Ce qui se passe pour `GET /soirees/8`*

### Les middlewares, dans l'ordre

Slim exécute en premier le middleware ajouté en dernier. Dans `config/middlewares.php` :

1. **CorsMiddleware** : ajoute les en-têtes `Access-Control-*` pour les origines autorisées, répond 204 aux requêtes `OPTIONS`. Il est à l'extérieur pour que même les réponses d'erreur aient les en-têtes CORS.
2. **ErrorMiddleware** avec `ProblemJsonErrorHandler` : transforme toute exception en réponse JSON avec le bon code HTTP.
3. **RoutingMiddleware** : trouve l'action à appeler.
4. **BodyParsingMiddleware** : décode le JSON des requêtes POST.

Deux middlewares sont posés seulement sur des **groupes de routes** (`config/routes.php`) : **AuthMiddleware** lit le jeton `Authorization: Bearer …`, le vérifie et place l'utilisateur connecté dans la requête (sinon 401) ; **AdminMiddleware** refuse ensuite tout compte qui n'a pas le rôle `ADMIN` (403). L'espace client passe par le premier, le backoffice par les deux.

## 6.3 Endpoints disponibles

| Route | Paramètres | Action Slim |
|---|---|---|
| `GET /` | — | `AccueilAction` |
| `GET /spectacles` | `date` (AAAA-MM-JJ), `style` (slug), `page` (≥ 1), `size` (1 à 50), tous facultatifs et cumulables | `GetSpectaclesAction` |
| `GET /spectacles/{id}` | id entier positif | `GetSpectacleAction` |

| Route | Paramètres | Action Slim |
|---|---|---|
| `GET /soirees/{id}` | id entier positif | `GetSoireeDetailAction` |
| `GET /styles` | — | `GetStylesAction` |
| `GET /dates` | — | `GetDatesAction` |
| `GET /soirees` · `GET /pass` | — | `GetSoireesAction` · `GetPassAction` |
| `POST /auth/register` | `email`, `nom`, `prenom`, `mot_de_passe` (8 à 72 caractères) ; renvoie 201 et un jeton | `InscriptionAction` |
| `POST /auth/signin` | `email`, `mot_de_passe` ; renvoie le jeton et le compte | `ConnexionAction` |
| `GET /compte` (connecté) | — | `GetCompteAction` |
| `POST /commandes` (connecté) | `lignes` : 1 à 20, chacune `type`, `soiree_id` ou `pass_id` et `soirees`, `categorie`, `quantite` (1 à 10) ; jamais de prix | `PostCommandeAction` |
| `GET /commandes`, `/commandes/{id}`, `/commandes/{id}/billets`, `/billets` (connecté) | id de commande UUID | `GetMesCommandesAction`, `GetCommandeAction` … |
| `POST /commandes/{id}/paiement` (connecté) | `numero`, `expiration` (MM/AA), `cryptogramme` | `PayerCommandeAction` |
| `GET /billets/{token}` | jeton UUID v4 | `GetBilletAction` |
| `/admin/…` (administrateur) | `GET jauges`, `GET referentiels`, `POST soirees`, `POST soirees/{id}/spectacles` | `GetJaugesAction`, `PostSoireeAction` … |

### GET /spectacles?style=metal&size=1


```
{
  "type": "collection",
  "count": 1, "total": 7, "page": 1, "size":
1,
  "filtres": { "date": null, "style": "metal"
},
  "spectacles": [ {
    "spectacle": {
      "id": 9, "titre": "Fer Rouge",
      "date": "2027-06-07", "horaire":
"20:30",
      "image": "img/spectacles/metal-1.webp",
      "style": { "slug": "metal",
                 "libelle": "Métal" } },
    "links": {
      "self":   { "href": "/spectacles/9" },
      "soiree": { "href": "/soirees/4" } } }
],
  "links": {
    "self":  { "href": "/spectacles?
style=metal&page=1&size=1" },
    "first": { "href": "…page=1…" },
    "next":  { "href": "…page=2…" },
    "last":  { "href": "…page=7…" } }
}
```


### GET /soirees/8 (extrait)


```
{
  "type": "resource",
  "soiree": {
    "id": 8, "nom": "Nuit métal",
    "thematique": "Soirée métal",
    "date": "2027-06-11", "heure_debut":
"19:30",
    "lieu": { "id": 2, "nom": "Zénith de
Nancy",
              "adresse": "rue du Zénith, …" },
    "tarifs": { "normal": 3000, "reduit": 2000
},
    "jauge": 5000, "places_restantes": 1350,
    "complete": false,
    "spectacles": [ {
      "id": 19, "titre": "Abyssal Forge",
      "artistes": [ "Abyssal Forge" ],
      "description": "Riffs massifs…",
      "style": { "slug": "metal", … },
      "horaire": "20:00",
      "images": [ { "url": "…", "alt": "…" }
],
      "video": "https://www.youtube-
nocookie.com/embed/…"
    }, … ]
  },
  "links": { "self": { "href": "/soirees/8" },
    "spectacles": { "href": "/spectacles?
date=2027-06-11" } }
}
```

### Erreurs

```
GET /spectacles?date=2027-02-31&page=0

400 application/problem+json
{
  "type": "https://nrv.net/erreurs/requete-
invalide",
  "title": "Requête invalide",
  "status": 400,
  "errors": {
    "date": "format attendu : AAAA-MM-JJ",
    "page": "entier superieur ou egal a 1
attendu"
  }
}
```

| Code | Cas |
|---|---|
| 400 | donnée invalide, avec un message par champ (date impossible, quantité hors bornes, carte mal formée) |
| 401 | jeton absent, faux ou expiré ; identifiants incorrects (en-tête `WWW-Authenticate: Bearer`) |
| 402 | paiement refusé par la banque (rien n'est enregistré) |
| 403 | route du backoffice appelée sans le rôle administrateur |
| 404 | ressource ou route inconnue ; aussi pour la commande d'un autre client (on ne révèle pas qu'elle existe) |
| 405 | méthode non autorisée (en-tête `Allow`) |
| 409 | e-mail déjà utilisé, commande déjà payée, places insuffisantes (avec le détail par soirée) |
| 422 | règle métier non respectée (numéro de carte inexistant, carte expirée, soirée complète) |
| 500 | erreur interne, sans aucun détail en production |

La sécurité (injections, validation, en-têtes) est détaillée dans la partie 8.

## 6.4 Commande et paiement : une seule transaction

La commande est créée « en attente » avec des prix **relus en base** (le panier n'envoie que des identifiants et des quantités). Le paiement (`PayerCommande`) se fait ensuite en entier dans une transaction SQL : au premier refus, tout est annulé.

1. **La commande est verrouillée** (`SELECT … FOR UPDATE`) : un deuxième paiement envoyé en même temps (double clic) attend la fin du premier, puis voit qu'elle est payée (409).
2. `Commande::payer()` vérifie dans le domaine qu'elle est en attente et que la carte n'est pas expirée.
3. **Les soirées sont verrouillées**, dans l'ordre de leur id (deux paiements ne peuvent pas se bloquer mutuellement) ; `Soiree::reserver()` refuse s'il ne reste pas assez de places.
4. La banque (simulée) est débitée seulement à ce moment : on ne débite jamais une commande qui ne pourra pas être servie.
5. Les places vendues, le statut payé (avec les 4 derniers chiffres de la carte seulement) et un billet par place (jeton UUID v4) sont enregistrés.

```
// PayerCommande::executer (extrait)
$commande = $this->transaction->executer(function () use (…): Commande {
    $commande = $this->commandes->trouverPourPaiement($commandeId);   // FOR UPDATE
    if ($commande === null || !$commande->appartientA($utilisateurId)) {
        throw CommandeIntrouvableException::pour($commandeId);         // 404
    }
    $commande->payer($carte, $this->horloge->maintenant());           // 409 si deja payee
    $places = $commande->placesParSoiree();
    $soirees = $this->soirees->verrouiller(array_keys($places));      // FOR UPDATE, par id
    foreach ($places as $soireeId => $nombre) {
        $soirees[$soireeId]->reserver($nombre);                        // 409 si plus de places
    }
    $this->banque->debiter($carte, $commande->total());               // 402 si refusee
    …
});
```

> **Vérifié en conditions réelles**
>
> Sur l'API locale, avec 8 processus PHP en parallèle :
>
> - la même commande de 2 billets payée **5 fois en même temps** : 1 paiement accepté, 4 refus 409 ; exactement 2 billets émis et 2 places comptées ;
> - **6 clients** qui paient en même temps les **2 dernières places** de « Blues et soul » : 2 acceptés, 4 refus 409 ; la soirée finit à 750 / 750.

## 6.5 Tests unitaires

**159 tests, 302 assertions, tous verts** (PHPUnit 11), dont 110 ajoutés au début de l'itération 2. Ils testent le noyau métier sans base de données ni Docker et couvrent **97 % des lignes** du domaine et des cas d'usage (mesure `--coverage-text`).

| Suite | Tests | Ce qui est vérifié |
|---|---|---|
| `tests/Domain` | 81 | soirée de 2 ou 3 spectacles, réservation et jauge, tarifs, un style par spectacle, pagination ; **Pass** sur 3 soirées distinctes ; **Commande** : prix venant de la soirée et du pass, un pass prend une place dans chacune de ses soirées, paiement une seule fois, carte expirée refusée, pas de billet avant le paiement, un billet par place avec sa référence et son jeton, 10 billets au plus par ligne ; **CarteBancaire** : algorithme de Luhn, expiration, seuls les 4 derniers chiffres gardés ; utilisateur (e-mail, rôle), billet (soirées, jeton) |
| `tests/Application` | 56 | filtres et pagination ; inscription (e-mail déjà pris, mot de passe trop court, haché), connexion (même refus pour un e-mail inconnu et un mauvais mot de passe) ; commande créée avec les prix du serveur, places insuffisantes, soirée inconnue ; paiement : places réservées et billets émis, carte refusée ou numéro faux → **rien n'est enregistré ni débité**, second paiement refusé, commande d'un autre client introuvable ; jauges et recettes ; ajout de soirées au backoffice |
| `tests/Infrastructure` | 22 | validation des formulaires de compte et du backoffice (images du site seulement), jeton JWT (signature, expiration, émetteur), middlewares 401 et 403 |

> **Faux adaptateurs (tests/Application/Mock)**
>
> Chaque port de sortie a un faux en mémoire : dépôts ( `InMemoryCommandeRepository` …), banque qui accepte ou refuse ( `FausseBanque` ), transaction ( `FausseTransaction` ), horloge fixe, générateur d'identifiants prévisible, hacheur et jeton factices. Un cas d'usage est ainsi testé seul, en quelques millisecondes.

Lancer les tests : `docker compose exec php composer test` (ou `vendor/bin/phpunit --testdox` pour le détail de chaque test).

Contrôle supplémentaire : en cassant volontairement la règle de jauge dans `Soiree::reserver()`, le test correspondant échoue bien. Le code respecte PSR-1 et PSR-4 (vérifié sur les 209 fichiers de `src` et `tests` : un espace de noms par dossier, une classe par fichier portant son nom, `declare(strict_types=1)` partout).

# 7 Frontend réalisé

Le site est un ensemble de pages HTML statiques qui interrogent l'API avec `fetch` et affichent les réponses avec des templates Handlebars. Il n'utilise aucun framework : du JavaScript natif en modules ES, une feuille Sass compilée et une feuille d'impression.

## 7.1 Pages et parcours

| Page | Contenu | Fonctionnalités |
|---|---|---|
| `index.html` | programme : filtres date et style cumulables, compteur de résultats, grille de cartes, pagination ; les filtres et la page sont gardés dans l'URL (lien partageable, bouton « retour » du navigateur) | F1, F3, F4, F5, E1 |
| `soiree.html?id=…` | détail d'une soirée : thématique, date, lieu, spectacles (horaire, style, artistes, description, galerie d'images, vidéo), encart d'achat avec places restantes et sous-total | F2, F6, E4 |
| `panier.html` | lignes du panier modifiables (− / + / supprimer), total, vider le panier ; composition d'un Pass 3 jours ; validation de la commande (connexion demandée si besoin, puis retour au panier) | F7, E2, E4, F8 |
| `connexion.html` · `inscription.html` · `compte.html` | formulaires de compte avec un message sous chaque champ en erreur ; retour à la page d'origine après la connexion ; fiche du compte et déconnexion | F8 |
| `paiement.html?commande=…` | barre d'étapes, récapitulatif de la commande (prix du serveur), carte bancaire, puis confirmation avec un lien par billet | F8, F9 |
| `billet.html` · `mes-billets.html` | billet par son jeton (public) ou tous les billets d'une commande, avec QR code et feuille d'impression ; liste de mes commandes et de leurs billets | F10 |
| `admin.html` · `admin-programme.html` | tableau des jauges et recettes, trié et mis en évidence ; ajout d'une soirée avec ses spectacles ou d'un spectacle à une soirée (administrateur seulement) | F11, E3 |

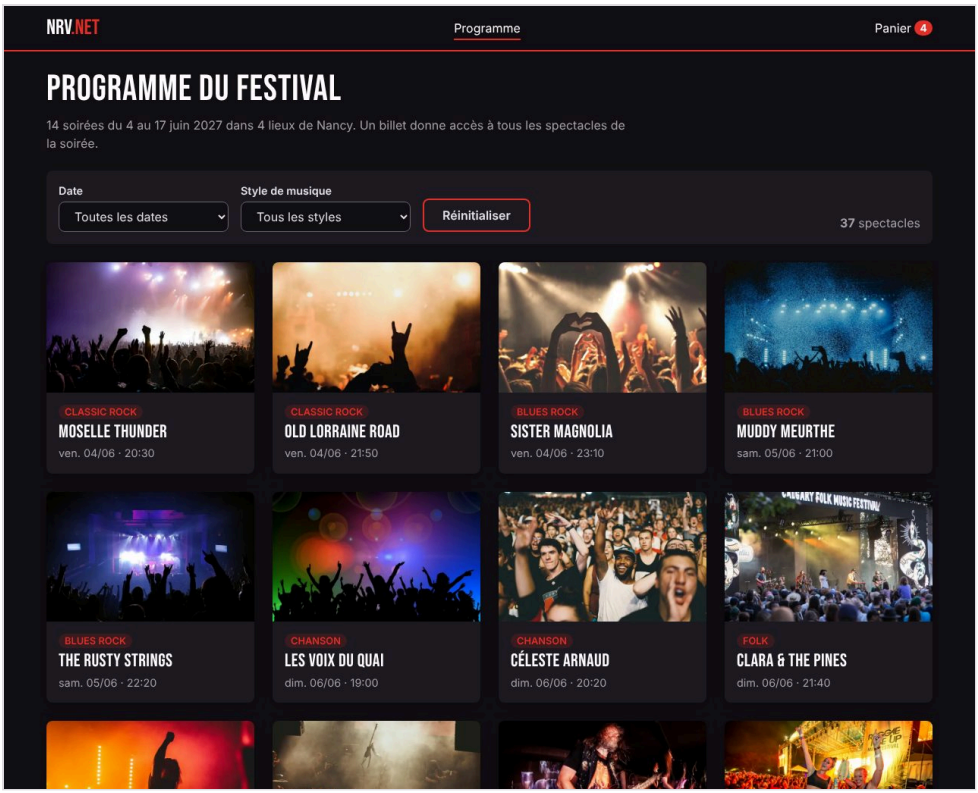



*Figure 20 · Programme sur ordinateur (données réelles de l'API)*

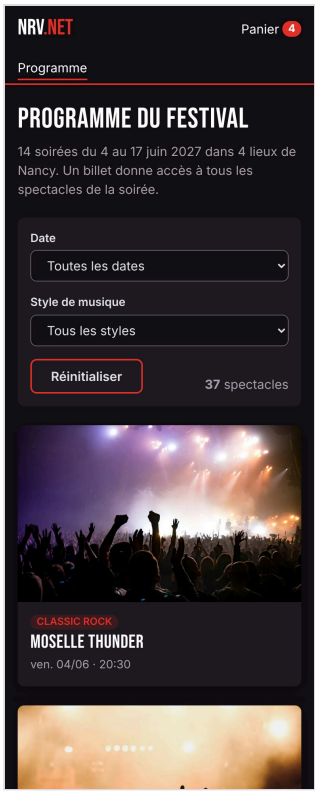



*Figure 21 · Sur téléphone*

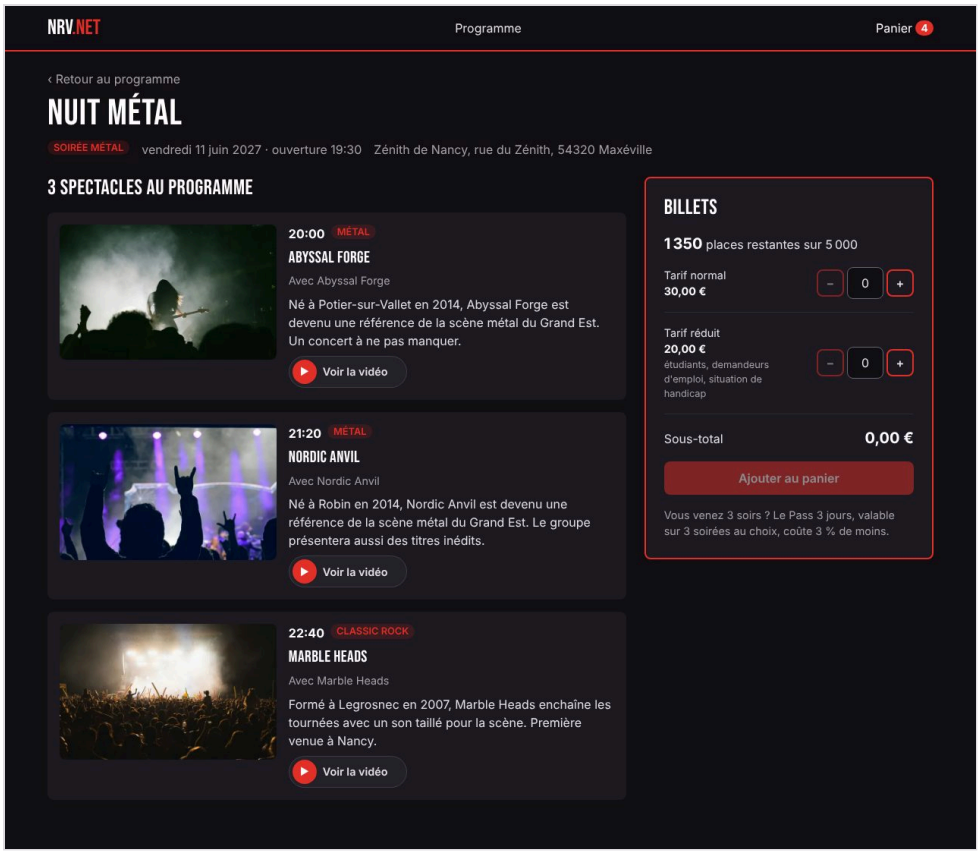



*Figure 22 · Soirée « Nuit métal » : 3 spectacles, vidéo chargée au clic, encart d'achat*

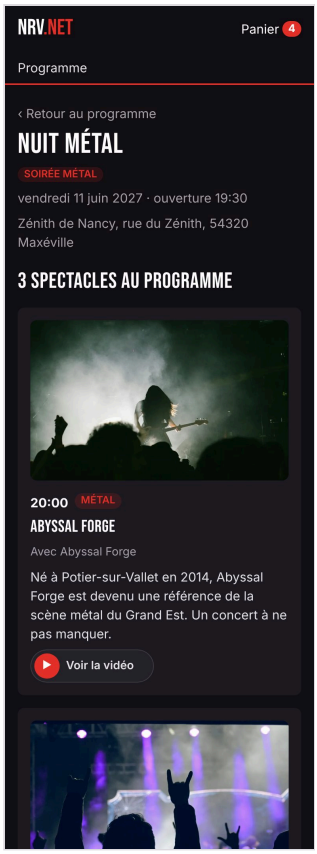



*Figure 23 · Sur téléphone*

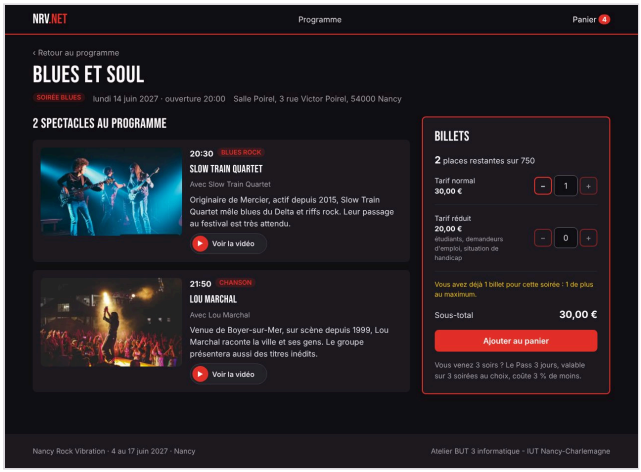



*Figure 24 · Soirée à 2 places : le + se bloque et un message explique pourquoi (E4)*

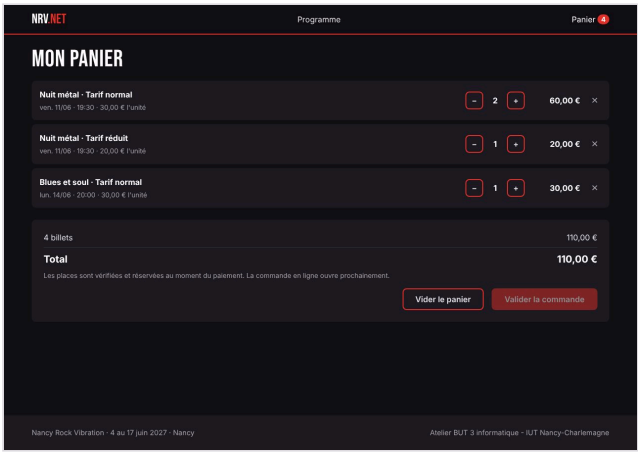



*Figure 25 · Panier : quantités modifiables et total (F7, E2)*

### Écrans de l'itération 2

Les écrans Connexion, Paiement, Billets et Backoffice, et la composition du Pass 3 jours, sont présentés avec leurs captures en partie 9. La navigation s'adapte à la session : « Connexion » ou « Mon compte », « Mes billets » une fois connecté, « Backoffice » pour un administrateur.

## 7.2 Organisation du code JavaScript

| Module | Rôle |
|---|---|
| `config.js` | adresse de l'API, déduite de celle du serveur (même nom d'hôte, port 8080) : rien à changer pour le déploiement |
| `api.js` | un seul point d'appel à `fetch`, qui ajoute l'en-tête `Authorization` pour les routes protégées ; transforme une erreur `application/problem+json` en `ErreurApi` (code HTTP, titre, détails) ; lit l'id dans un lien HATEOAS |
| `templates.js` | charge les fichiers `templates/*.hbs`, les compile une seule fois (cache) et enregistre les helpers de formatage |
| `format.js` | prix en centimes → « 30,00 € », dates et horaires en français (`Intl`) |
| `ui.js` | chargement, messages d'erreur avec bouton « Réessayer », notification après un ajout au panier |
| `programme.js` · `soiree.js` · `panier-page.js` | le code propre à chaque page |
| `panier.js` | le panier, gardé dans le `localStorage` ; il est synchronisé entre onglets grâce à l'événement `storage` |
| `session.js` | la session (jeton JWT et compte) dans le `localStorage` ; fermée dès que le jeton expire ; pages de retour après la connexion contrôlées par une liste blanche |
| `entete.js` · `formulaire.js` | en-tête commun selon la session ; envoi des formulaires, erreurs de l'API affichées sous le bon champ, bouton occupé pendant l'envoi |
| `connexion.js` · `paiement.js` · `billet.js` · `admin-jauges.js` … | le code des pages de l'itération 2 ; `qr.js` dessine le QR code d'un billet en SVG |

### Du clic à l'affichage : un filtre sur le programme

1. Le choix d'un style met à jour l'URL (`history.pushState`) sans recharger la page.
2. La requête précédente, si elle n'est pas finie, est annulée (`AbortController`) : un résultat ancien ne peut pas écraser le nouveau.
3. `GET /spectacles?style=metal&page=1` renvoie la page de résultats et ses liens `next` / `prev`.
4. Chaque carte pointe vers sa soirée grâce au lien HATEOAS `links.soiree` ; la pagination n'affiche « suivant » que si l'API fournit `links.next`.
5. Le template `spectacles.hbs` produit le HTML, inséré dans la grille ; le compteur annonce le résultat aux lecteurs d'écran (`aria-live`).

```
{{!-- templates/carte-spectacle.hbs : toute la carte mène à la soirée (F3) --}}
<article class="carte-spectacle">
  <img class="carte-spectacle__image" src="{{image}}" alt="{{titre}} sur scène"
       width="800" height="500" loading="lazy">
  <div class="carte-spectacle__corps">
    <span class="etiquette">{{style.libelle}}</span>
    <h3 class="carte-spectacle__titre">
      <a class="carte-spectacle__lien" href="soiree.html?id={{soireeId}}">{{titre}}</a></h3>
    <p class="carte-spectacle__infos"><time datetime="{{date}}T{{horaire}}">{{dateCourte date}} ·
{{horaire}}</time></p>
  </div>
</article>
```

Les cas d'erreur sont tous gérés à l'écran : API injoignable, filtre invalide dans l'URL (400), soirée inconnue (404), aucun résultat, soirée complète (bouton d'achat désactivé).

## 7.3 Sass : architecture et bibliothèque d'outils

```
src/scss/
├── abstracts/  variables, mixins
├── base/       reset, typographie
├── layout/     conteneur, en-tête, pied
├── components/ boutons, cartes, filtres…
├── pages/      soirée, panier, billet…
├── main.scss   → css/main.css
└── print.scss  → css/print.css
```

`abstracts/` ne produit aucun CSS : il ne contient que des outils. Les fichiers sont reliés avec `@use` (modules Sass) et les classes suivent la convention BEM : `.carte-spectacle__titre`, `.etiquette--complet`.

```
// abstracts/_variables.scss
$breakpoints: ('tablette': 600px, 'bureau': 1024px);

// abstracts/_mixins.scss
@mixin mq($nom) {
  @if not map.has-key($breakpoints, $nom) {
    @error "breakpoint inconnu : #{$nom}";
  }
  @media (min-width: map.get($breakpoints, $nom)) {
    @content;
  }
}

@mixin flex($justify: flex-start, $align: center,
            $gap: $espace-m, $wrap: nowrap) {
  display: flex;
  justify-content: $justify;
  align-items: $align;
  gap: $gap;
  flex-wrap: $wrap;
}
```

### La bibliothèque d'outils ( `abstracts/` )

| Outil | Usage |
|---|---|
| variables | couleurs par rôle ( `$couleur-accent` , `$couleur-surface` …), polices, échelle d'espacements ( `$espace-xs` à `$espace-xl` ), rayon, ombre, largeur maximale, map des points de rupture |

| Outil | Usage |
|---|---|
| `@mixin mq($nom)` | media query « mobile d'abord » à partir d'un nom ; une faute de frappe arrête la compilation (`@error`) |
| `@mixin flex(…)` | une ligne flex en une instruction, avec des valeurs par défaut : `@include flex(space-between, center, $espace-m, wrap)` |
| `@mixin conteneur` | largeur maximale centrée avec marges intérieures |
| `@mixin focus-visible` | contour jaune visible au clavier seulement, appliqué à tous les éléments interactifs |
| `@mixin visuellement-cache` | texte lu par les lecteurs d'écran mais invisible (« Page 2 » dans la pagination, numéro des photos de la galerie) |

### Sélecteurs avancés utilisés

| Sélecteur | Pourquoi |
|---|---|
| `.carte-spectacle__lien::after { inset: 0 }` | le lien du titre couvre toute la carte : un seul lien par carte, donc une seule tabulation au clavier |
| `&:hover, &:focus-within` | la carte se soulève aussi quand son lien a le focus clavier |
| `.spectacle-detail { & + & { … } }` | espace seulement entre deux spectacles qui se suivent |
| `[aria-pressed='true']`, `[aria-current='page']`, `[aria-invalid='true']` | le style suit l'état d'accessibilité : miniature choisie, page courante, champ en erreur ; pas de classe d'état à maintenir en double |
| `> span[aria-hidden='true']` | la pastille « lecture » du bouton vidéo, distincte du texte lu par le lecteur d'écran |
| `.pass__case:checked + .pass__carte`, `:disabled +`, `:focus-visible +` | sélecteur du Pass 3 jours : la vraie case reste utilisable au clavier mais est cachée (mixin `visuellement-cache`), la carte qui la suit affiche son état et dessine la case avec `::before`, sans JavaScript |
| `li + li::after` avec `counter()` | étapes de commande numérotées et reliées par un trait, sans balisage en plus |
| `::-webkit-inner-spin-button` | retire les flèches natives des champs nombre, remplacées par les boutons – / + |

Les sélecteurs restent ceux vus en cours : combinateurs, attributs, pseudo-classes et pseudo-éléments.

## 7.4 Grid et Flexbox : quand et pourquoi

Règle suivie : **Grid** quand la mise en page est en deux dimensions ou quand des colonnes doivent s'aligner, **Flexbox** pour aligner des éléments sur une ligne qui peut passer à la ligne.

| Composant | Choix | Justification et adaptation |
|---|---|---|
| Grille des spectacles | Grid | `repeat(auto-fill, minmax(16rem, 1fr))` : le nombre de colonnes dépend de la place (1 sur téléphone, 2 sur tablette, 4 sur ordinateur) sans aucune media query, et toutes les cartes d'une ligne ont la même hauteur |
| Page soirée | Grid | une colonne sur mobile ; `2fr 1fr` sur ordinateur (spectacles \| achat), l'en-tête occupe toute la largeur ( `grid-column: 1 / -1` ) |
| Fiche d'un spectacle | Grid | image au-dessus du texte, puis côte à côte dès la tablette ( `minmax(12rem, 2fr) 3fr` ) |
| Billet, paiement, backoffice | Grid | zones fixes (billet \| talon, récapitulatif \| carte, indicateurs en 3 colonnes) ; sélecteur du pass en `repeat(auto-fill, minmax(15rem, 1fr))` : 1 colonne sur téléphone, 4 sur ordinateur |
| En-tête et navigation | Flexbox | logo \| liens \| panier avec `space-between` ; passe à la ligne si l'écran est étroit |
| Barre de filtres | Flexbox | `flex: 1 1 12rem` : les champs s'empilent sur téléphone et restent en ligne sur ordinateur ; le compteur est poussé à droite ( `margin-left: auto` ) |
| Lignes du panier, encart d'achat | Flexbox | libellé \| quantité \| prix sur une ligne ; sur téléphone le libellé prend toute la largeur ( `flex-basis: 100%` ) et les commandes passent dessous |
| Sélecteur − [n] +, pagination, infos de la soirée | Flexbox | alignement vertical centré, espacement par `gap` |

## 7.5 Adaptation au terminal

| Aspect | Ce qui est fait |
|---|---|
| Images | une photo par spectacle, recadrée en 16/10 et enregistrée en WebP en deux tailles (640 et 1200 px, environ 30 et 70 Ko) ; `srcset` et `sizes` : le navigateur charge la petite version sur un écran classique et la grande sur un écran haute densité ; `width` / `height` dans le HTML et `aspect-ratio: 16 / 10` pour réserver la place (pas de saut de page au chargement) ; `object-fit: cover` ; `loading="lazy"` ; le lecteur YouTube n'est chargé qu'au clic |
| Mise en page | mobile d'abord, deux points de rupture (600 px et 1024 px) ; encart d'achat collant ( `position: sticky` ) seulement sur ordinateur ; aucun défilement horizontal à 390 px (vérifié) |
| Typographie | tailles en `rem` (respectent le réglage du navigateur) ; titre principal de 2,25 rem sur mobile à 3 rem sur ordinateur ; polices Inter et Bebas Neue avec polices système de secours |
| Interaction | zones tactiles d'au moins 36 px ; états `:hover` et `:focus-visible` distincts ; clavier numérique sur mobile ( `inputmode="numeric"` ) ; animations coupées si l'utilisateur le demande ( `prefers-reduced-motion` ) ; lien « Aller au contenu » |
| Impression | `print.css` ( `media="print"` ) : menus et boutons masqués, noir et blanc, billet jamais coupé entre deux pages |

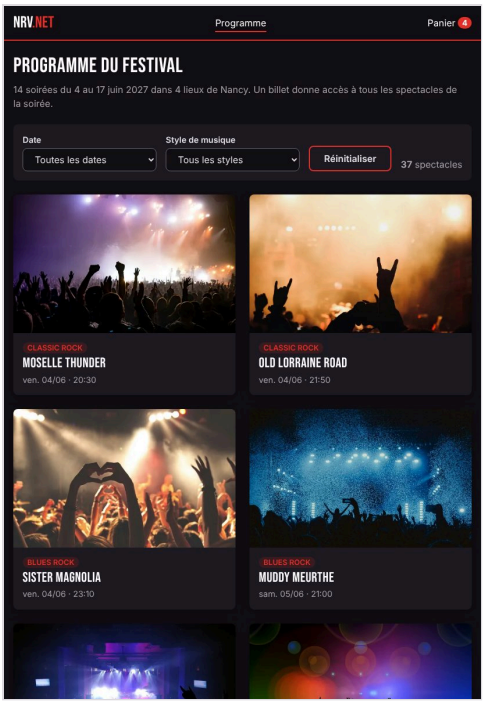



*Figure 26 · Tablette (820 px) : 2 colonnes*

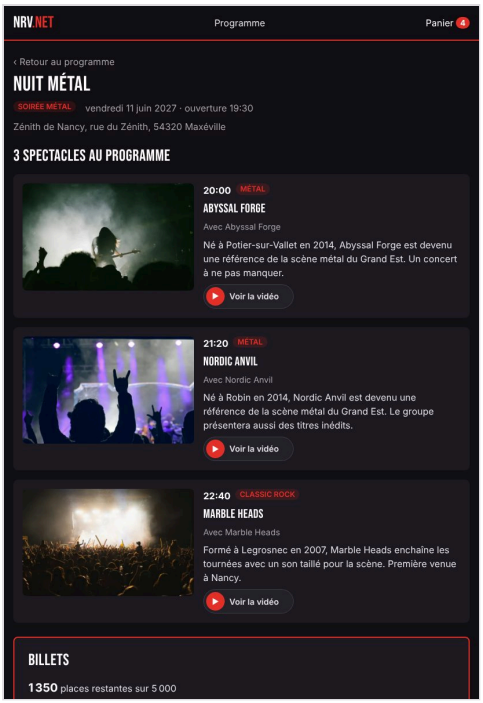



*Figure 27 · Soirée sur tablette : image à côté du texte*

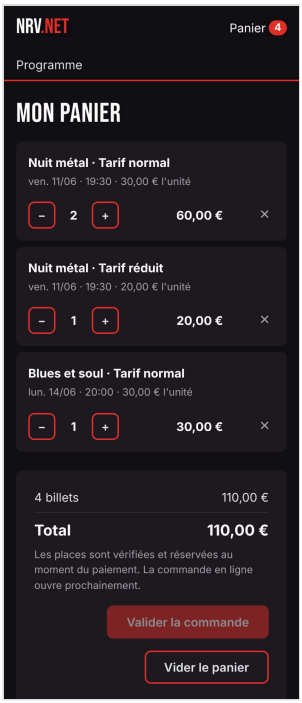



*Figure 28 · Panier sur téléphone*

## 7.6 Qualité HTML et vérifications

- **HTML valide W3C** : 0 erreur au validateur officiel (Nu Html Checker) sur les 11 pages du site et sur les états rendus par le JavaScript ; la feuille CSS compilée est également valide.
- **Sémantique** : `header`, `nav`, `main`, `footer`, une liste `ul` de cartes `article`, `time datetime` pour les dates, `output` pour les quantités calculées, `label` sur chaque champ, un seul `h1` par page et une hiérarchie de titres continue.
- **Accessibilité** : zones mises à jour annoncées (`aria-live`, `aria-busy`), boutons − / + et pagination nommés pour les lecteurs d'écran, focus visible, contraste du texte sur fond sombre.
- **Tests dans un vrai navigateur** (Playwright, derrière le serveur nginx du projet) : **117 vérifications, toutes réussies**, en trois séries : itération 1 (47 : F1 à F7, E1, E2, E4, erreurs 400 et 404, retour du navigateur), comptes et pass (35), commande, paiement, billets et backoffice (35 : F8 à F11, E3, carte refusée, places prises entre-temps, impression, téléphone). Chaque série vérifie aussi l'absence d'erreur JavaScript, de violation de la CSP et de défilement horizontal sur téléphone.

# 8 Sécurité

Le site et l'API sont protégés contre les attaques courantes du web : injection SQL, injection de HTML ou de script (XSS), intégration dans un site piège (clickjacking), appels depuis un autre site, et, depuis l'itération 2, vol de compte, jeton falsifié, accès aux données d'un autre client et fraude sur la commande. Chaque protection a été vérifiée par des tests d'attaque, présentés en 8.6.

## 8.1 Injections SQL

Une injection SQL consiste à glisser du code SQL dans un paramètre (par exemple `?style=x' OR 1=1 --`) pour qu'il soit exécuté par la base. L'API s'en protège à deux niveaux.

### Requêtes préparées partout

- La requête SQL est envoyée à MariaDB **avant** les valeurs, avec des marqueurs (`:style`, `?`). Les valeurs arrivent ensuite séparément : elles sont toujours traitées comme des données, jamais comme du SQL.
- Les vraies requêtes préparées du serveur sont forcées (`PDO::ATTR_EMULATE_PREPARES => false`) ; les erreurs SQL lèvent une exception (`ERRMODE_EXCEPTION`).
- `LIMIT` et `OFFSET` sont liés en entiers (`PDO::PARAM_INT`).
- Le `WHERE` n'assemble que des morceaux de SQL écrits dans le code ; une valeur saisie n'y est jamais concaténée.
- Une liste d'identifiants (`IN (…)`) reçoit autant de `?` que de valeurs, toutes liées.

```
// PDOSpectacleRepository::rechercher (extrait)
$conditions = [];
$parametres = [];
if ($criteres->date() !== null) {
    $conditions[] = 'so.date_soiree = :date';
    $parametres['date'] = $criteres->date()->format('Y-m-d');
}
if ($criteres->style() !== null) {
    $conditions[] = 'st.slug = :style';
    $parametres['style'] = $criteres->style();
}
$where = $conditions === []
    ? '' : ' WHERE ' . implode(' AND ', $conditions);

$requete = $this->pdo->prepare(
    SpectacleHydrator::SELECT . $where
    . ' ORDER BY … LIMIT :limite OFFSET :decalage');
foreach ($parametres as $nom => $valeur) {
    $requete->bindValue($nom, $valeur, PDO::PARAM_STR);
}
$requete->bindValue('limite', $criteres->taille(), PDO::PARAM_INT);
$requete->bindValue('decalage', $criteres->decalage(), PDO::PARAM_INT);
$requete->execute();
```

> **Audit du code**
>
> Tous les appels à la base de `api/src` ont été relus : chacun passe par `prepare()`, aucun par `query()` ou `exec()` avec une donnée extérieure. Seul le script de données de test (`bin/seed.php`, lancé à la main, jamais exposé sur le web) utilise `exec()` et `query()`, sur du SQL fixe écrit dans le script (liste de tables en dur, comptages).

## 8.2 Validation des entrées

Avant d'arriver aux cas d'usage, chaque paramètre et chaque corps JSON est contrôlé par un validateur de l'adaptateur HTTP (`FiltreSpectaclesValidator`, `IdentifiantValidator`, `CompteValidator`, `CommandeValidator`, `ProgrammeAdminValidator`). Une valeur incorrecte est refusée avec un code **400** et un message par champ : elle n'atteint jamais la base.

| Paramètre | Règle |
|---|---|
| `date` | format `AAAA-MM-JJ` et date réelle (`checkdate` : le 31 février est refusé) |
| `style` | slug de 1 à 60 caractères en `[a-z0-9-]` : ni apostrophe, ni espace, ni point-virgule |
| `page`, `size` | chiffres uniquement, 6 au maximum ; `page` de 1 à 10 000, `size` de 1 à 50 |
| `{id}` | entier positif de 9 chiffres au maximum (pas de `0x08`, de `8 OR 1=1` ni de nombre géant) |
| tous | valeur simple obligatoire : un tableau (`?style[]=metal`) est refusé |
| compte | e-mail valide de 180 caractères au plus, nom et prénom de 80 au plus, mot de passe de 8 à 72 caractères ; un champ `role` envoyé est ignoré |
| commande, carte | 1 à 20 lignes, quantités de 1 à 10, catégorie `NORMAL` ou `REDUIT` ; carte de 13 à 19 chiffres, expiration `MM/AA`, cryptogramme de 3 ou 4 chiffres |
| backoffice | textes bornés, date et heures réelles, tarifs entiers ; une image doit être un fichier du site (`img/…`) : ni adresse externe, ni `javascript:`, ni `../` |

Les règles métier sont ensuite vérifiées une seconde fois par les entités du domaine (jauge, tarifs, nombre de spectacles), et la base ajoute ses propres contraintes (`CHECK`, clés étrangères, `UNIQUE`).

## 8.3 Côté site : XSS et en-têtes HTTP

| Risque | Protection |
|---|---|
| Injection de HTML ou de script (XSS) | Handlebars échappe toutes les valeurs (`{{titre}}` : `<` devient `<`) ; la syntaxe sans échappement `{{{ }}}` ne sert qu'au QR code, un SVG dessiné par le site à partir du jeton du billet ; hors templates, le texte est inséré avec `textContent` ou `createElement`, et `innerHTML` ne reçoit que du HTML produit par un template |
| Panier modifié dans le navigateur | chaque ligne relue du `localStorage` est contrôlée (type, catégorie, entiers bornés, 10 billets au maximum par ligne) ; une ligne invalide est ignorée. Les prix du panier sont indicatifs : le serveur les recalcule à la commande |

| Risque | Protection |
|---|---|
| Vidéo piégée | une iframe n'est créée que si l'adresse correspond à `https://www.youtube(-nocookie).com/embed/…` ; la version `nocookie` ne dépose pas de cookie avant la lecture |
| Redirection vers un autre site | le lien « Retour au programme » mémorisé n'est accepté que s'il commence par `index.html` ; après la connexion, la page de retour (`?retour=`) doit être une page du site de la liste blanche |
| Script ou ressource extérieure | en-tête **Content-Security-Policy** : scripts du site seulement, appels réseau vers le site et l'API seulement, iframes YouTube seulement, polices Google seulement, aucun plugin (`object-src 'none'`) |
| Clickjacking | `frame-ancestors 'none'` et `X-Frame-Options: DENY` : le site ne peut pas être affiché dans un cadre d'un autre site |
| Fichiers mal interprétés, fuite d'adresse | `X-Content-Type-Options: nosniff` ; `Referrer-Policy` (`no-referrer` pour l'API) |
| Informations sur le serveur | version de nginx masquée (`server_tokens off`), en-tête `X-Powered-By` de PHP retiré (`expose_php = Off`), erreur 500 sans message ni trace en production (`APP_ENV=prod`) |
| Appels depuis un autre site | CORS : l'API ne renvoie `Access-Control-Allow-Origin` que pour les origines listées dans `CORS_ORIGINS` |
| Accès à la base, dépendances | MariaDB n'écoute que sur la machine (`127.0.0.1`) ; dépendances sans faille connue (php-jwt 7, la version 6 étant touchée par la CVE-2025-45769 ; Slim ≥ 4.15.2 ; PHPUnit ≥ 11.5.50) |

La CSP autorise `'unsafe-eval'` pour les scripts : c'est nécessaire à `Handlebars.compile`, qui transforme les templates en fonctions dans le navigateur. Cela n'ouvre pas de faille XSS puisque les scripts extérieurs et les scripts écrits dans la page (`inline`) restent interdits. Précompiler les templates permettrait de retirer cette exception.

## 8.4 Comptes et jetons

| Risque | Protection |
|---|---|
| Vol de la base des mots de passe | hachage **bcrypt** ( `password_hash` , coût 12) : le mot de passe n'est jamais stocké ni renvoyé en clair ; le hash n'apparaît dans aucune réponse |
| Deviner les comptes existants | même réponse 401 et même message pour un e-mail inconnu et un mauvais mot de passe ; pour un e-mail inconnu, un hash factice est quand même vérifié pour que le refus prenne le même temps |
| Jeton falsifié | JWT signé en **HS256** avec un secret lu dans `.env` ; signature, algorithme, émetteur ( `iss` ) et expiration vérifiés à chaque requête : un jeton modifié (rôle changé en `ADMIN` ), non signé ( `alg: none` ), signé avec une autre clé ou expiré est refusé (401) |
| Devenir administrateur | le rôle n'est jamais lu dans la requête : il vient du jeton, que seule l'API sait signer, et `AdminMiddleware` le contrôle sur tout le backoffice (403) ; l'inscription crée toujours un compte client |
| Session oubliée | jeton valable 1 heure ; le site ferme la session dès que le jeton a expiré et renvoie vers la connexion sur un 401 |

## 8.5 Commande et paiement

| Risque | Protection |
|---|---|
| Prix modifié dans le navigateur | le corps de la commande ne contient aucun prix : le total est recalculé à partir des tarifs en base (un champ `prix` ajouté est ignoré) |
| Voir ou payer la commande d'un autre | chaque cas d'usage vérifie que la commande appartient au compte du jeton ; sinon 404, sans dire si elle existe ; identifiants de commande en UUID, impossibles à énumérer |
| Double paiement, survente | transaction avec verrouillage de la commande puis des soirées (6.4), et contrainte `CHECK` en base en dernier rempart |
| Fuite de la carte | numéro contrôlé (Luhn), jamais enregistré : seuls les 4 derniers chiffres sont gardés et renvoyés ; le cryptogramme n'est pas conservé |
| Billet deviné | jeton UUID v4 aléatoire ( `random_bytes` ) : la page publique d'un billet ne s'ouvre qu'avec ce jeton |

## 8.6 Tests d'attaque

Les requêtes suivantes ont été envoyées à l'API locale ; après les tentatives d'injection, le contenu de la base a été recompté et était identique. La série complète (42 vérifications, toutes réussies) est rejouée après chaque étape.

| Requête envoyée | Réponse |
|---|---|
| `/spectacles?style=x' OR 1=1 --` | 400 |
| `/spectacles?style=metal'; DROP TABLE soirees;--` | 400 |
| `/spectacles?date=2027-06-11' OR '1'='1` | 400 |
| `/spectacles?date=2027-06-11 UNION SELECT 1` | 400 |
| `/spectacles?page=1 OR 1=1` · `?size=10;DELETE FROM billets` | 400 · 400 |
| `/spectacles?page=-1` · `?page=99999999999999999999` · `?size=1000` | 400 · 400 · 400 |
| `/spectacles?style[]=metal` | 400 |
| `/soirees/8 OR 1=1` · `/soirees/1' UNION SELECT * FROM utilisateurs--` | 400 · 400 |
| `/soirees/0x08` · `/soirees/9999999999` | 400 · 400 |
| `/spectacles/<script>alert(1)</script>` | 404 |
| requête `OPTIONS` avec `Origin: https://evil.example` | 204 sans en-tête CORS |
| connexion avec `admin@nrv.net' --` | 401 |
| inscription avec `"role": "ADMIN"` | 201, compte CLIENT |
| `/admin/jauges` sans jeton · avec un jeton client | 401 · 403 |
| jeton `alg: none` · signé avec une autre clé · expiré · autre émetteur · rôle modifié à la main | 401 (×5) |
| `POST /commandes` avec `"prix": 1` · quantité −5 · 25 lignes | total 30,00 € · 400 · 400 |
| consulter, payer ou lister les billets de la commande d'un autre client | 404 (×3) |
| image de spectacle `javascript:alert(1)` au backoffice | 400 |
| soirée nommée `<img src=x onerror=alert(1)>` puis affichée sur le site | texte affiché, rien d'exécuté |

Côté site, les tests navigateur vérifient aussi qu'un `<img src=x onerror=…>` placé dans l'URL ou dans le panier s'affiche comme du texte, sans être exécuté, et qu'aucune violation de la CSP n'apparaît sur l'ensemble du parcours.

Limite connue : il n'y a pas encore de limitation du nombre d'essais de connexion ; le coût de bcrypt ralentit une attaque par force brute, une limitation par adresse IP est prévue pour la suite (partie 9).

# 9 Début de l'itération 2

L'itération 2 est planifiée sur le Trello en quatre étapes. Les étapes 5, 6 et 7 sont réalisées : un visiteur peut maintenant créer un compte, composer un Pass 3 jours, commander, payer, imprimer ses billets, et l'administratrice suit les ventes et complète le programme. L'étape 8 (finalisation et rendu) est en cours.

| Étape | Cartes | Membres | État |
|---|---|---|---|
| **5. Authentification et Pass** | authentification JWT, hachage et routes `/auth` · middleware qui réserve le backoffice aux administrateurs · route `GET /pass` · pages connexion et inscription · sélecteur du Pass 3 jours | Ilyass | Réalisé |
| **6. Panier et tunnel d'achat** | validation du panier (connexion requise) · création de commande · formulaire de paiement et confirmation · paiement simulé `POST /commandes/{id}/paiement` | Saad | Réalisé |
| **7. Billets et backoffice** | consultation des billets · billet imprimable ( `@media print` ) · jauges du backoffice · tableau de bord des jauges · ajout de spectacles et de soirées · écran des billets de l'utilisateur | Luca, Kamdine, Saad | Réalisé |
| **8. Finalisation et rendu** | tests unitaires PHPUnit du domaine · tests de sécurité · déploiement sur docketu · README final et dossier · dépôt final vendredi 20 h | Kamdine, Saad | Tests faits, rendu en cours |

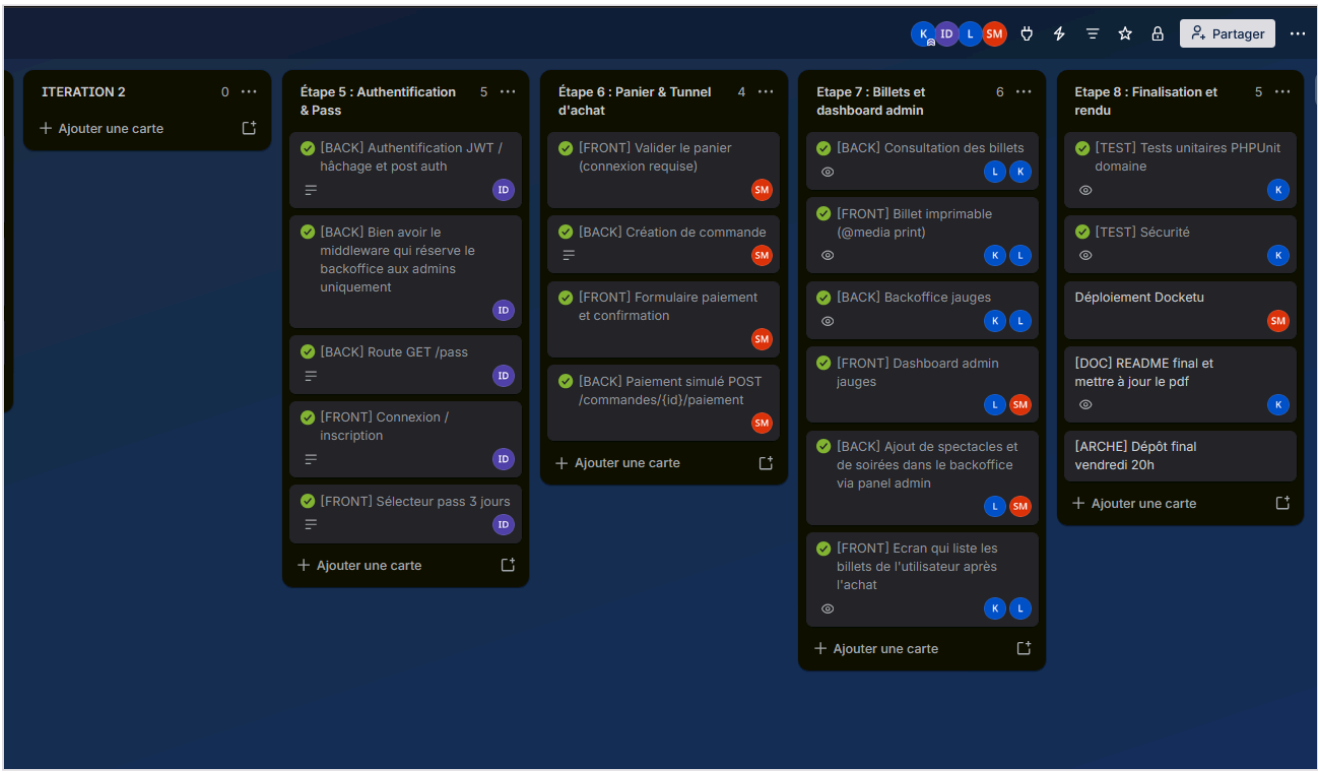



*Figure 29 · Tableau Trello de l'itération 2 (état au 23 octobre 2026)*

## 9.1 Étape 5 : comptes et Pass 3 jours

- **API** : cas d'usage `InscrireUtilisateur` et `AuthentifierUtilisateur`, ports `HacheurMotDePasseInterface` (bcrypt) et `JetonInterface` (JWT) ; `AuthMiddleware` et `AdminMiddleware` ; routes `/auth/register`, `/auth/signin`, `/compte`, `/pass`, `/soirees`.
- **Site** : pages Connexion, Inscription et Mon compte ; après la connexion, retour à la page de départ (le panier par exemple) ; composition du Pass 3 jours dans le panier : 3 soirées distinctes, soirées complètes grisées, compteur, prix du pass comparé à 3 billets séparés.

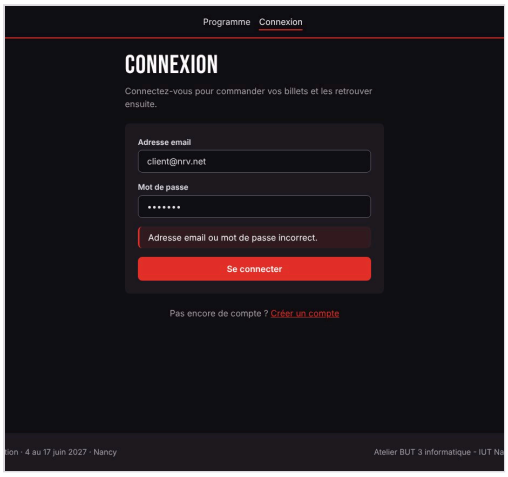



*Figure 30 · Connexion refusée : un seul message, sans dire quel champ est faux*

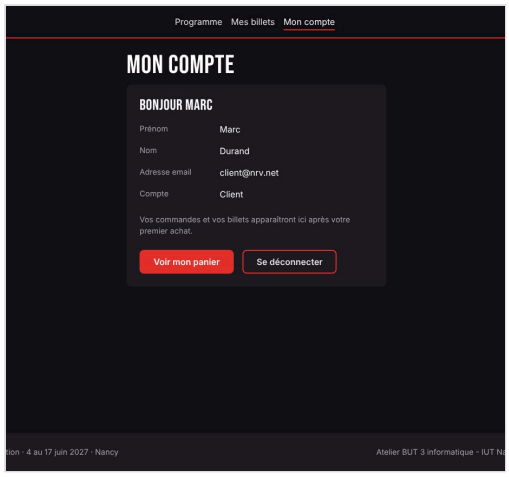



*Figure 31 · Mon compte*

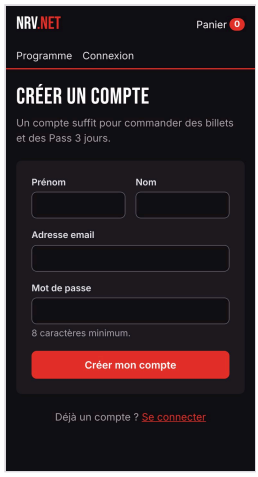



*Figure 32 · Inscription sur téléphone*

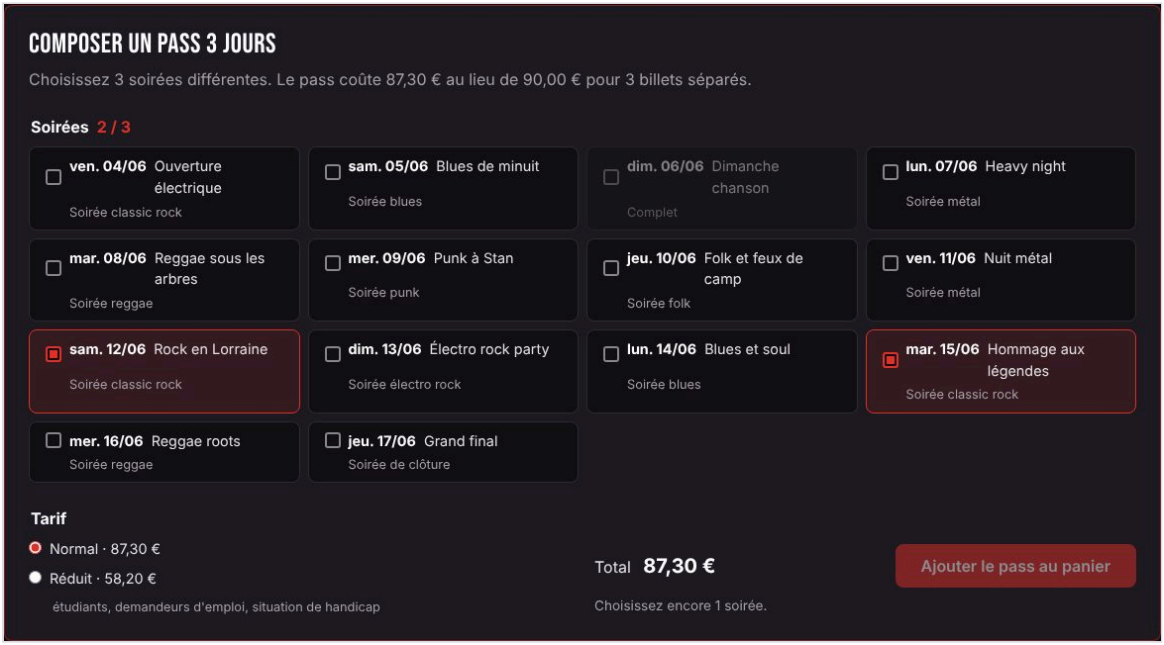



*Figure 33 · Composer un Pass 3 jours : 2 soirées choisies sur 3, la soirée complète est grisée, la case active a le focus clavier*

## 9.2 Étape 6 : commande et paiement

Le panier est validé (connexion demandée si besoin), la commande est créée par `POST /commandes` avec les prix du serveur, puis payée sur la page Paiement. Les refus sont expliqués à l'écran : numéro de carte faux, carte expirée, paiement refusé par la banque (la carte de test `4000 0000 0000 0002` est toujours refusée, `4242 4242 4242 4242` acceptée), soirée devenue complète entre-temps (avec son nom). Le fonctionnement de la transaction est détaillé en 6.4.

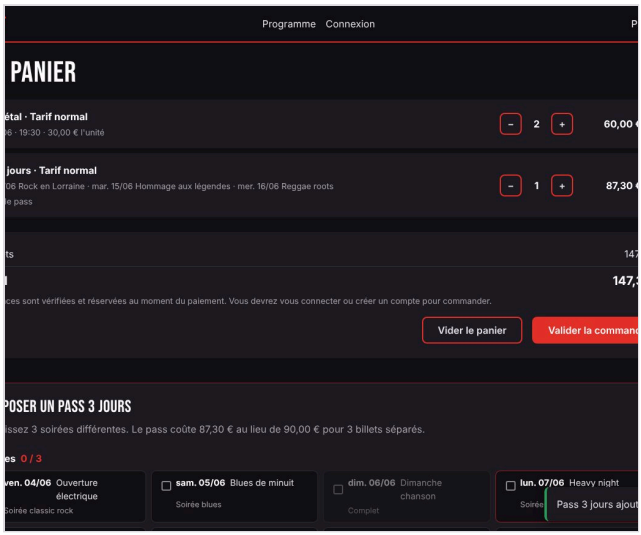



*Figure 34 · Panier : 2 billets et un Pass 3 jours*

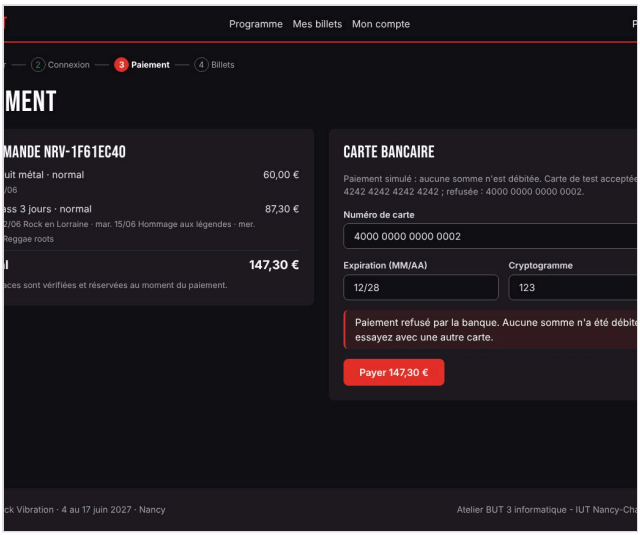



*Figure 35 · Paiement : récapitulatif, carte, et refus de la banque (rien n'est débité)*

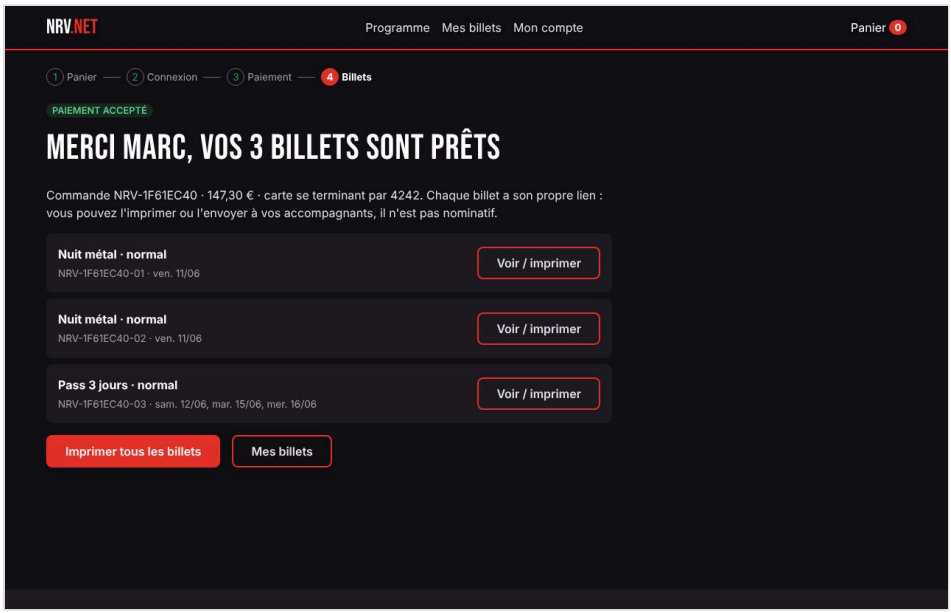



*Figure 36 · Confirmation : un lien par billet, carte se terminant par 4242*




*Figure 37 · Sur téléphone*

## 9.3 Étape 7 : billets et backoffice

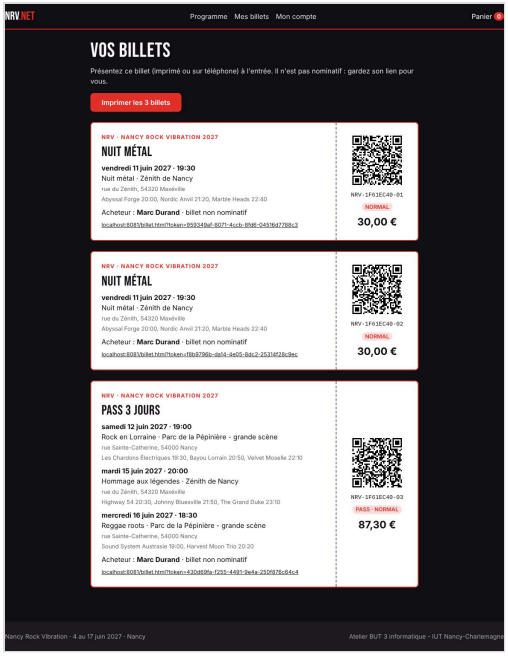



*Figure 38 · Les 3 billets d'une commande à l'écran*

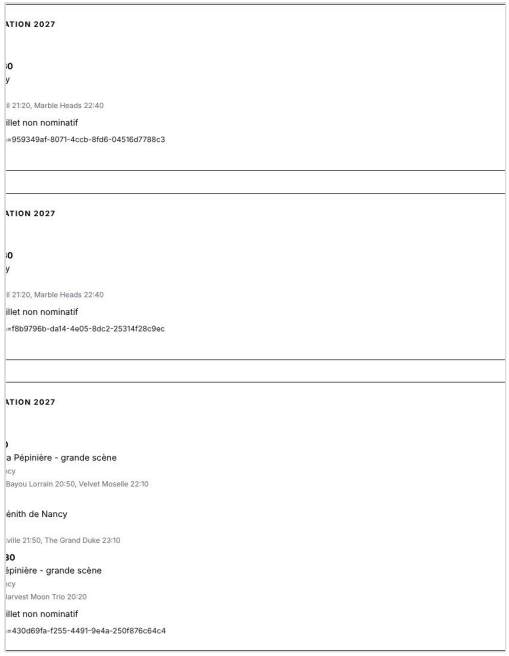



*Figure 39 · À l'impression : noir et blanc, sans menu*




*Figure 40 · Sur téléphone*

Chaque billet porte le nom de l'acheteur, sa référence (`NRV-971B1AF0-01`), la ou les soirées, la catégorie, le prix et un QR code de son adresse publique (F10). « Mes billets » regroupe les commandes du compte.

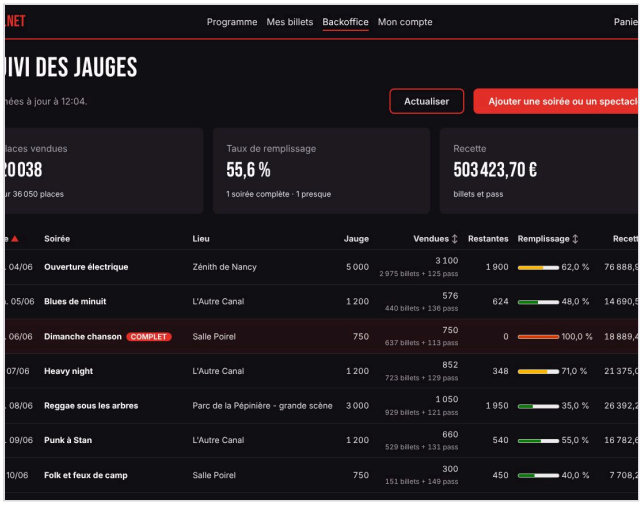



*Figure 41 · Backoffice : jauges, places vendues et recette (F11)*

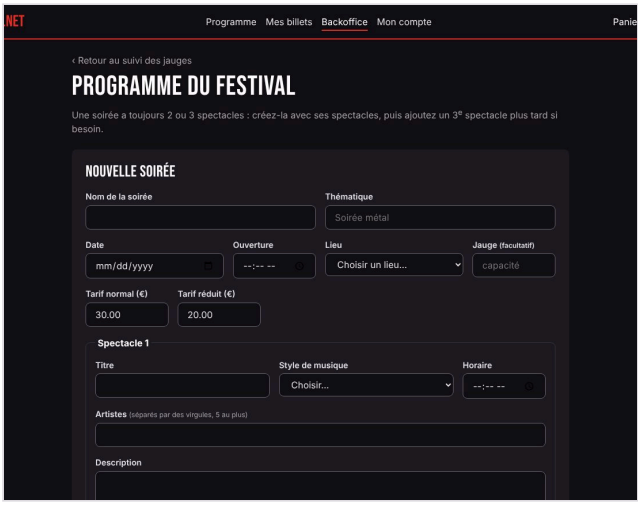



*Figure 42 · Ajout d'une soirée et de ses spectacles (E3)*

## 9.4 Répartition du travail

Comme à l'itération 1, chacun a pris en charge une étape sur sa branche, puis les branches ont été réunies et testées ensemble avant d'être fusionnées sur `main`.

| Membre | Part de l'itération 2 |
|---|---|
| Ilyass DGHISSI | étape 5 : inscription et connexion (bcrypt, JWT), middlewares connecté et administrateur, route du Pass, pages de compte et sélecteur du Pass 3 jours |
| Saad MOUBINE | étape 6 : validation du panier, création de commande, paiement simulé, pages paiement et confirmation ; tableau de bord et ajout au programme avec Luca ; déploiement sur docketu |
| Luca VIDIRI | étape 7 : billets, billet imprimable, jauges et tableau de bord, ajout de soirées et de spectacles, écran « Mes billets » |
| Kamdine HASSANZADEH | étape 7 avec Luca (billets, jauges, mes billets) ; étape 8 : tests unitaires du domaine, tests de sécurité, README et dossier de rendu |

Les messages de commit reprennent les cartes du Trello (`[back] commande et paiement simule`, `[test] tests unitaires de la commande et du paiement`, `[front] validation du panier, paiement et confirmation`…), comme à l'itération 1.

## 9.5 Suite de l'itération 2

- **Déploiement** : mettre à jour la version en ligne sur docketu avec la version finale de `main`, puis y rejouer les tests navigateur.
- **Refonte visuelle** : harmoniser les écrans de l'itération 2 avec ceux de l'itération 1 (typographie, espacements, états des composants), en gardant l'identité noir et rouge, le responsive et la validité W3C.
- **Sécurité** : limiter le nombre d'essais de connexion par adresse IP ; précompiler les templates Handlebars pour retirer `'unsafe-eval'` de la CSP.
- **Rendu** : README final, archive de rendu.

# 10 Conclusion

NRV.net couvre aujourd'hui l'ensemble du sujet : toutes les fonctionnalités, **F1 à F11 et E1 à E4**, fonctionnent de bout en bout, de la base de données jusqu'au site.

L'itération 1 a posé les fondations : l'analyse du sujet, le modèle du domaine, la base de données, le contrat d'API et les maquettes, puis l'environnement Docker et la partie consultation (programme, filtres, pagination, détail des soirées, panier). Ces choix de départ ont tenu : l'itération 2 a pu ajouter les comptes, le Pass 3 jours, la commande, le paiement, les billets et le backoffice sans revenir sur la structure, simplement en ajoutant des cas d'usage, des ports et des adaptateurs.

Côté serveur, l'architecture hexagonale et l'injection de dépendances ont rendu le noyau métier testable seul : 159 tests unitaires le vérifient sans base de données. Les règles importantes du sujet sont appliquées par le serveur et non par le navigateur : les prix sont recalculés, les places sont contrôlées et réservées au moment du paiement dans une transaction, et un billet n'est accessible que par son jeton. Côté site, les pages restent en HTML valide, mises en forme avec Sass, Grid et Flexbox, alimentées par `fetch` et Handlebars, et utilisables sur téléphone, tablette, ordinateur et à l'impression.

Le travail en équipe s'est appuyé sur le Trello, une carte par tâche et une branche par membre, ce qui a permis d'avancer en parallèle sur les étapes 5, 6 et 7. Il reste à finaliser le déploiement sur docketu et le dépôt de l'archive ; la refonte visuelle des nouveaux écrans, une limitation des essais de connexion et la précompilation des templates sont les prochaines améliorations envisagées.